



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka Bengaluru - 560064



SMART COMPETENCY DIAGNOSTIC AND PROFILE SCORE CALCULATOR

A PROJECT REPORT

Submitted by

FAUZAN BIN JAVID- 20221CDV0002

TILAK HEJIB- 20221CDV0007

BHARAT KAUSHIK- 20221CDV0044

Under the guidance of,

Dr. Akshatha Y

Assistant Professor-Selection Grade

BACHELOR OF TECHNOLOGY

IN

**COMPUTER SCIENCE AND TECHNOLOGY
(DevOps)**

PRESIDENCY UNIVERSITY

BENGALURU

DECEMBER 2025



PRESIDENCY UNIVERSITY

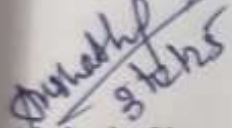
Private University Estd. in Karnataka State by Act No. 41 of 2013
Italguppa, Rajanukunte, Yelahanka Bengaluru - 560064

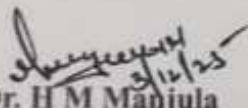


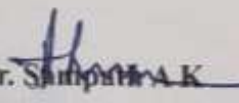
PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

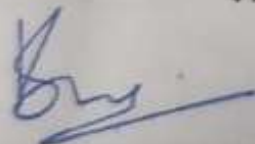
Certified that this report "Smart Competency Diagnostic and Profile Score Calculator" is a bonafide work of "Fauzan Bin Javid (20221CDV0002), Tilak Hejib (20221CDV0007), Bharat Kaushik (20221CDV0044)", who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND TECHNOLOGY**, DevOps during 2025-26.


Dr. Akshatha Y
Project Guide
SCS
Presidency University


Dr. H M Manjula
Program Project
Coordinator
PSCS
Presidency University

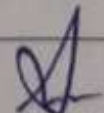
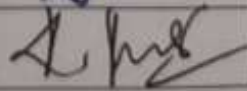

Dr. Geetha A
School Project
Coordinators
PSCS
Presidency University


Dr. Pravinthraja S
Head of the Department
PSCS
Presidency University


Dr. Shakkeera L
Associate Dean
SCS
Presidency University


Dr. Duraipandian N
Dean
PSCS & PSIS
Presidency University

Name and Signature of the Examiners

Sl. no.	Name	Signature	Date
1	Ms. Shweta Singh		5/12/25
2	Dr. Rukin Koushik		5/12/25

PRESIDENCY UNIVERSITY
PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND
ENGINEERING
DECLARATION

We the students of final year B.Tech in **COMPUTER SCIENCE AND ENGINEERING** at Presidency University, Bengaluru, named **Fauzan Bin Javid, Tilak Hejib, and Bharat Kaushik**, hereby declare that the project work titled "**Smart Competency Diagnostic and Profile Score Calculator**" has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in **COMPUTER SCIENCE AND TECHNOLOGY** during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Fauzan Bin Javid USN: 2022ICDV0002

Tilak Hejib USN: 2022ICDV0007

Bharat Kaushik USN: 2022ICDV0044

Fauzan

Tilak

Bharat

PLACE: BENGALURU

DATE: 3 DECEMBER 2025

ACKNOWLEDGEMENT

For completing this project work, we have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved Chancellor, Pro-Vice Chancellor, and Registrar for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide **Dr. Akshatha Y**, Assistant Professor, Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Pravinthraja S**, Professor, Head of the Department, Presidency School of Computer Science and Engineering Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, **Dr. Shakkeera L**, Associate Dean, Presidency School of computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K**, and **Dr. Geetha A**, PSCS Project Coordinators, **Dr. H M Manjula**, Program Project Coordinator, Presidency School of Computer Science and Engineering, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

FAUZAN BIN JAVID

TILAK HEJIB

BHARAT KAUSHIK

ABSTRACT

Smart Competency Diagnostic and Profile Score Calculator is web-based, serverless, and should be used for multi-format technical testing, real-time code execution, and AI-driven competencies. The system comprises a product developed in React as a front-end function, Supabase based on the code executed in sandbox sandboxed with Judge0, and the language model is available to scale the testing to be fit for the users abilities and development potential. The systems architecture and implementation comprise the main features, such as the quiz assessment, coding assessment with manually set auto-grading based on test cases, AI-generated career and college course suggestions, and complete Row Level Security (RLS) for all users. The results show that the system is a successfully deployed serverless educational web application with a front end that is administrative-friendly, load times of less than 2 seconds and customized solutions as visible in the code execution window. The application incorporates database level privacy measures to ensure that only front-end exposures are not made at the cost of front-end exposure. The project demonstrates that serverless architectures in educational technology are indeed attainable and can serve to achieve high-functioning simplicity and cost-efficiency.

TABLE OF CONTENT

Sl. No.	Title	Page No.
	Declaration	iv
	Acknowledgement	iv
	Abstract	iv
	List of Figures	iv
	List of Tables	ix
	Abbreviations	X- iv
1.	Introduction	1
	1.1 Background	1
	1.2 Statistics of Project	1
	1.3 Prior Existing Technologies	2
	1.4 Proposed Approach	2
	1.5 Objectives	3
	1.6 SDGs	3
	1.7 Overview of Project Report	4
2.	Literature review	6
3.	Methodology	10
	3.1 Introduction to Methodology	10
	3.2 V Model Component of the Hybrid Methodology	11
	3.2.1 Requirement Phase	11
	3.2.2 System and Functional Design	11
	3.2.3 Detailed and Unit Design	12
	3.2.4 Implementation Phase	12
	3.2.5 Verification Stages	12
	3.3 Agile Component of the Hybrid Methodology	13
	3.3.1 Sprint Based Development	13

	3.3.2 Agile Tools and Processes	13
	3.4 Mapping Project Stages to the Hybrid Methodology	13
	3.5 System Diagrams and Visual Methodology Elements	16
	3.5.1 System Block Diagram	16
	3.5.2 Workflow Execution Diagram	16
	3.5.3 Data Flow Diagram	16
	3.6 Verification and Validation Activities	17
	3.7 Module Interaction	18
	3.8 Summary of Methodology	19
4.	Project management	20
	4.1 Project Timeline	20
	4.2 Risk Analysis	22
	4.3 Project Budget	23
5.	Analysis and Design	27
	5.1 Requirements	27
	5.2 Block Diagram	28
	5.3 System Flow Chart	31
	5.4 Choosing Software Tools and Frameworks	33
	5.5 Designing Units	34
	5.5.1 Code Execution Unit	35
	5.5.2 AI Mentor Unit	36
	5.5.3 Recommendation Unit	36
	5.6 Standards	36
	5.7 Mapping to AI Agent Architecture Layers	37
	5.8 Domain Model Specification	39
	5.9 Communication Model	40
	5.10 AI Deployment Level	40
	5.11 Functional View	41
	5.12 Mapping Deployment Level with Functional View	42
	5.13 Operational View	42
	5.14 Other Design Aspects	43

6.	Hardware, Software and Simulation	44
	6.1 Hardware	44
	6.2 Software Development Tools	45
	6.3 Software Code	45
	6.3.1 Core Agent Logic	46
	6.3.1 Memory Manager	46
	6.3.2 Workflow Engine	46
	6.3.3 Extended Tool Interface (MCP Tool)	46
	6.3.4 Safety Layer	47
	6.4 Simulation	47
7.	Evaluation and Results	49
	7.1 Test Points	49
	7.2 Test Plan	52
	7.3 Test Result	54
	7.4 Insights	56
8.	Social, Legal, Ethical, Sustainability and Safety Aspects	58
	8.1 Social Aspects	58
	8.2 Legal Aspects	59
	8.3 Ethical Aspects	59
	8.4 Sustainability Aspects	60
	8.5 Safety Aspects	60
	8.6 Conclusion	61
9.	Conclusion	62
	References	64
	Base Paper	65

List of Figures

Figure	Caption	Page no
Fig 1.1	Sustainable Development Goals	4
Fig 3.1	Hybrid Methodology Overview (V-Model + Agile Loop)	11
Fig 3.2	Three-Tier Architecture	16
Fig 3.3	Detailed Workflow	17
Fig 3.4	Process Flowchart	17
Fig 4.1	Project Planning Timeline	21
Fig 4.2	Project Implementation Timeline	21
Fig 5.1	Functional Block Diagram	29
Fig 5.2	System Flow Chart	32
Fig 5.3	Sequence Diagram	35
Fig 5.4	Domain Model Specification Diagram	39
Fig 5.5	Functional View	40
Fig 5.6	Operational View	41
Fig 5.6	Operational View	38
Fig 5.7	Deployment Level with Functional View	38
Fig 5.8	Operational View	39

List of Tables

Table	Caption	Page no
Table 1.1	Sustainable Development Goals Alignment	4
Table 2.1	Summary of Literature Reviews	8
Table 3.1	Mapping of Smart Competency Diagnostic Project Stages to Hybrid Methodology	13
Table 4.1	Project Planning Timeline	21
Table 4.2	Project Implementation Timeline	21
Table 4.3	PESTEL Analysis	23
Table 4.4	Another PESTEL Analysis	24
Table 4.5	Project Phase Risk Matrix	29
Table 4.6	Project Budget	26
Table 5.1	Summary of Requirements	28
Table 5.2	Comparing Backend and AI Frameworks Options	33
Table 5.3	Mapping Project Components With AI Agent Architecture	38
Table 5.4	Mapping Functional View To Deployment Artifacts	42
Table 7.1	MCQ Assessment Results	54
Table 7.2	Code Execution Performance	55
Table 7.3	Feature / First-Token Latency	56

ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BaaS	Backend-as-a-Service
CUDA	Compute Unified Device Architecture
DB	Database
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
LMS	Learning Management System
MCQ	Multiple Choice Question
MCP	Model Context Protocol
ML	Machine Learning
RLS	Row-Level Security
SDG	Sustainable Development Goal
SPA	Single Page Application
SSE	Server-Sent Events
UI	User Interface
TP	Test Point

CHAPTER 1

INTRODUCTION

We operate in the tech world on continuous, accurate, scalable ways to test and demonstrate someone's technical competency and provide tailored professional guidance. Meanwhile, classic assessment systems are severely stretched by various fundamental deficiencies: no real-time feedback, an inflexibility that doesn't fit what a user can do — and a foundational fragmentation that disconnects hands-on coding validation from structured career planning. Together, these challenges inhibit efficient large-scale talent spotting for organizations and prevent true personalized, optimized career paths for people. Introducing the Smart Competency Diagnostic and Profile Score Calculator. This is a solution specifically designed to bridge those gaps. It's a single, web-based platform; think of this as an integration of multi-format tech assessments with live code execution and AI-powered adaptive mentorship. On the technical side, this is a modular and scalable architecture on a modern React frontend with a Supabase Backend-as-a-Service that runs the logic layer and a secure back-end with PostgreSQL database, with heavy lifting by external APIs—Judge0 does the sandboxed code execution, Lovable AI Gateway means we can use LLMs.

1.1 Background

Scalability, security, and personalization are issues facing the bulk of modern applications for EdTech and recruitment. Assessment Silos: Systems such as HackerRank are great at coding but not rigorous with the concepts. Conversely, standard LMS platforms like Canvas excel at theory but lack sandboxed code execution environments [1]. Guidance Not Available: While users take tests and sometimes get only a raw score, there's little "what next?" guidance. Despite promising outcomes when integrating AI into education [3], few systems leverage structured LLM outputs to produce a clear career roadmap. Security & Scalability: Old-school server-based architectures are quite expensive to maintain and scale. Moreover, for the security of running untrusted user code, the use of extensive sandboxing is required.

.

1.2 Statistics

Such a system is necessary, as evident from the outputs of the deployment undertaken in the pilot phase of the project:

- User Base: Over 100 users interacted with the platform during the pilot testing phase which revealed

that the need for unified assessment tools was apparent in real-time.

- **Performance:** Serverless architecture response was 2 seconds dashboard load time and 50ms database query latency, which made the user experience seamless despite being under load.
- **AI Response:** The AI Mentor module generated a first-token latency of less than 1 second, simulating body language speeds, which are required with a successful mentorship.
- **Reliability of Implementation:** The system can be made use of code submissions in four languages (Python, Java, C++, JavaScript) with a 100 per cent sandbox isolation.

1.3 Prior Existing Technologies

The competitive landscape should be critically analyzed to discover the opportunity of innovativity and positioning of the proposed solution in the right position. However, although different platforms can cover certain parts of the technical assessment and education, there is still a huge gap on an integrative solution. The constraints of the previous technologies could be divided into three different categories.

Online Evaluation Systems: HackerRank and LeetCode are the leaders in the industry who have positioned themselves as the best places to find practical problems of coding. Automated testing of algorithmic skills is their major strength. They are however very limited as they usually do not have modules to measure theoretical knowledge on fundamental subjects in computer science and do not provide customized career advice that is available as an integrated system where users have to look elsewhere to find their mentors.

Conventional LMS Platform: Canvas and Moodle are the most popular examples of Learning Management Systems that are actively used by academic institutions to provide course materials and offer the theoretical examinations. Although they have the capability to execute real time code in a secure and sandboxed environment, in essence they lack the ability to do so. Moreover, they do not commonly include AI-assisted recommendations of career or courses.

AI Tutoring Systems: AI tutors have provided much promise when it comes to customized learning because of the emergence of AI tutors, which are modeled after systems such as ChatGPT. These systems are good in conversational guidance and clarification. But they tend to be in use without a formal evaluation system that allows formalizing the competency of a user or monitoring their advancement on a known curriculum.

Overall, although all of these technologies contribute to a bit of the education puzzle, none of them is able to cover the entire board with a practical implementation of codes and an AI-driven mentorship approach. This fragmentation presents a definitive niche in the market of an integrated, holistic platform

and preconditions the innovative approach to architecture as offered by this project.

1.4 Proposed Approach

The competitive aspects must be critically examined to find out the innovativity and positioning of the solution in the right place. It still faces a massive gap in the integrative solution even if different platforms can cover some of the technical assessment and education aspects. From the perspective of existing technologies, limited data can be categorized into three types.

Online Evaluation Systems: HackerRank and LeetCode are the industry leaders who have established themselves as the foremost platforms to discover coding practical problems at this stage of development for coding skills, and their major strength lies in the automated test of algorithmic skills--their best assets. But they are extremely limited, with virtually no ability to assess theory around basic fields in computer science without building practical knowledge modules, or offer personalized career advice for individuals who want to look elsewhere to find their mentors integrated system for advice on computer courses as an integrated job guidance system.

The Common LMS Platform: Canvas and Moodle are the most commonly used learning management system and both are highly utilized by academic institutions, for example the learning and theoretical examination. Though they can run code in real time, in privacy and in isolation, they're not really able to. In addition, such solutions do not typically offer AI-based career or course recommendations.

AI Tutoring Systems: While AI tutors are modeled after systems like ChatGPT, they offer much promise for individualized education. They work well as conversational aids and clarifications. But they are mostly used in isolation, unable to provide a formalized testing system to indicate the competence of a user or to track their progress on a known curriculum.

While all of these technologies contribute to a tiny piece of that education puzzle, none of them can adequately address that problem across the board through a practical code implementation and AI Mentorship. This fragmentation creates a clear niche for the integrated holistic market and sets the stage for the innovative style of architecture as found in this project.

1.5 Objectives

- To develop a scalable serverless infrastructure including user authentication, data storage and a business logic without the physical servers [5].

- Deploy a protected execution engine based on Judge0 for sandboxing and grading code written in multiple languages (Python, Java, C++, JavaScript) [6].
- To execute Database-Level Security by Row-Level Security (RLS) policies to prevent a data breach and ensure users only have access to their records [7].
- To build in AI-based mentorship which provides a clear career progression & source guidance and be offered in the form of classes that will be implemented based on performance reviews.
- Develop an integrated dashboard to avoid confusion by guiding the user on their progress through visualizing performance metrics (accuracy, time spent occupation, skill distribution).

1.6 SDGs

SDG	Alignment with Project	Supporting References
SDG 4: Quality Education	Provides accessible, high-quality technical assessment and personalized mentorship to students regardless of location.	[3][4]
SDG 8: Decent Work & Economic Growth	Directly links skill competency to job market trends and Indeed job search URLs, facilitating employment.	[9]
SDG 9: Industry, Innovation & Infrastructure	Demonstrates the use of modern serverless infrastructure and AI to innovate in the EdTech sector.	[5][8]

Table 1.1 Sustainable Development Goals Alignment



Fig 1.1 Sustainable Development Goals

1.7 Overview of Project Report

This report is split into nine chapters. Chapter 1 will provide the background, motivation and the objectives of the project. Chapter 2 discusses the literature review of the ten research articles that informed this work. The hybrid methodology and system architecture diagrams are described in Chapter 3. In Chapter 4 we set up the project work, timelines and risk analysis. Chapter 5 presents the analysis with design, schema, database management and inter-component relationships. Chapter 6 supplies the vocabulary on hardware/software requirements and their implementation details. Chapter 7 introduces you to assessment measures and outcomes. In Chapter 8 social, ethical and legal is examined, followed by a conclusion in Chapter 9.

CHAPTER 2

LITERATURE REVIEW

Within the scope of automated testing, serverless cloud computing and machine learning in education, the creation of the so-called Smart Competency Diagnostic and Profile Score Calculator is based on a wide-ranging research. In this chapter, ten important works are reviewed which contributed to the architectural and functional decisions of this project.

[1] A Survey Of Automated Assessment Techniques For Programming Assignments (Ala-Mutka): The author discusses automated assessment systems that can assess correct code as well as style. It stressed that although automatic grading diminishes instructor workload and complexity, conventional grading systems tend not to provide theoretical evaluation or whole-student feedback. This understanding guided the development of an MCQ-based theoretical assessment and practice coding dilemma combined in a single competency-based diagnostic tool. It concluded that although these systems proved effective in delivering content, their greatest weakness is the absence of integrated environments where users can conduct a real-time code execution and receive custom AI advice. It provided a rationale for embedding specific technical assessment tools as part of the learning flow itself, with no need to use external plugins.

[2] A Review on Learning Management Systems: In-Depth Analysis (Alkhuraiji): The review concludes that while these systems are effective in terms of content delivery and administrative tracking, their greatest weakness is that they rarely have integrated environments for real-time code execution and personalized AI guidance. This provided the justification needed for our custom solution to embed specific technical assessment tools directly into the learning workflow rather than relying on disparate external plugins.

[3] Performance of ChatGPT on USMLE: Potential for AI-Assisted Medical Education (Kung et al.): Using LLMs for high-stakes assessments, the investigation revealed complex reasoning capabilities, but the implementation of these must be cautious for successful tutoring. This guidance and context-aware prompting on the part of LLMs was the basis for the "AI Mentor" module, ultimately justifying LLM use for guidance in the relevant environment of technical education. A systematic review found that structured prompts lead to consistent educational outputs from AI models. This has a direct bearing on the development of the course-suggestions and career-suggestions edge functions, wherein stringent

JSON schemas are imposed on prompts that keep parsing errors to a minimum and offer actionable recommendations.

[4] Prompt Engineering Systematic Review (mapped from your structured-prompt paragraph): A systematic review found that structured prompts lead to consistent educational outputs from AI models. This has a direct bearing on the development of the course-suggestions and career-suggestions edge functions, wherein stringent JSON schemas are imposed on prompts that keep parsing errors to a minimum and offer actionable recommendations.

[5] Cloud Programming Simplified: A Berkeley View on Serverless Computing (Jonas et al.): This presented serverless computing as an answer to conventional server management's scalability and maintenance overhead. This play heavily contributed toward the choice to run a serverless architecture (Edge Functions of Supabase), so that the application could be auto-scaled while requiring users to provide additional requests by clicking or dragging users through the application.

[6] Serverless Computing: Current Trends and Open Problems (Lee et al.): This literature described a trend toward "Backend-as-a-Service (BaaS)". This led to the decision to be using Supabase, a system that combines database and user-based authentication services, in order to avoid developing much time on the software side of things and lets application logic play a priority.

[7] Judge0: Open-Source Online Code Execution System (Judge0): Judge0 is an extensive API for code compilation and execution inside the sandbox. In isolation using Docker containers, to prevent an executing malicious code is a common task in our paper. Judge0 is used as the core-execution engine that supports time and memory restrictions for a safe handling of student code submissions.

[8] Row-Level Security in PostgreSQL: Design Patterns (Brown): This work involved studying advanced database security patterns -- Row-Level Security (RLS) in particular. RLS moves security from the application layer into the database kernel. It prevents data leaks in case of frontend-level compromise. It is the security model underneath that ensures users can only read their own assessment results. It is in this way, that they never leave it to outsiders.

[9] AI-Powered Career Guidance Systems: A Survey (Kumar and Chen): The survey found that while it recognized personalized career roadmaps improves outcomes for students, few existing systems connect skills assessment results directly to career advice. The proposed effort fills

this gap by feeding assessment scores into the AI model to offer hyper-personalized job role recommendations.

[10] Bringing the Web up to Speed with WebAssembly (Haas et al.): This work considered WebAssembly performance for browsers in the modern world. This technology is the foundation of the quality of performance of the Monaco Editor used on the current project to make the system to perform the low level of performance directly, such as with IDE-like high performance directly within the browser without any issue of latency.

A performance comparison of the WebAssembly brought up to speed Haas et al. cover the performance aspects of WebAssembly in modern browsers [10]. This technology forms the basis of functionality of the current web based code editors such as the Monaco Editor in our project. Knowledge of these abilities meant we were able to provide an IDE-like high-performance kind of experience straight through the browser without any hint of latency.

Summary of Literatures reviewed

Sl. No	Article Title, Author, Year	Methods	Key Features	Merits	Demerits
1	Automated Assessment Approaches, Ala-Mutka, 2005 [1]	Survey of assessment tools	Analysis of code grading techniques	Highlights efficiency of auto-grading ²	Notes lack of theoretical assessment integration ³
2	Evaluating LMS, Alkhuraiji et al., 2023 [2]	Comparative analysis	Review of Moodle/Canvas capabilities	Establishes baseline for ed-tech features ⁴	Highlights lack of real-time coding tools in standard LMS ⁵
3	Performance of ChatGPT, Kung et al., 2023 [3]	Experimental testing	LLM performance on standardized tests	Validates AI for complex reasoning ⁶	Warnings on hallucination risks ⁷

Sl. No	Article Title, Author, Year	Methods	Key Features	Merits	Demerits
4	Prompt Engineering, Zhang et al., 2023 [4]	Systematic Review	Techniques for structured AI outputs ⁸	Improves AI reliability ⁹	Requires complex implementation logic ¹⁰
5	Cloud Programming Simplified, Jonas et al., 2019 [5]	Architectural analysis	Serverless scalability models ¹¹	Justifies removal of server management ¹²	High dependency on cloud providers ¹³
6	Judge0 System, Šimurina, 2023 [6]	System implementation	Sandboxed code execution APIs ¹⁴	Secure, multi-language support ¹⁵	Latency dependency on API response ¹⁶
7	Row-Level Security, Brown, 2023 [7]	Security analysis	Database-level access control ¹⁷	Superior data isolation ¹⁸	Complex policy definitions required ¹⁹
8	Serverless Computing Trends, Lee et al., 2022 [8]	Technology survey	BaaS and Function-as-a-Service trends ²⁰	Reduces development overhead ²¹	Potential cold-start latency issues ²²
9	AI Career Guidance, Kumar et al., 2023 [9]	Survey	AI applications in career mapping ²³	Links skills to job market trends ²⁴	Data freshness can be a challenge ²⁵
10	WebAssembly, Haas et al., 2017 [10]	Performance benchmarking	Browser-based computation ²⁶	Enables desktop-class web apps ²⁷	Browser compatibility constraints ²⁸

Table 2.1 Summary of Literature reviews

CHAPTER 3

METHODOLOGY

The design and development of both the Smart Competency Diagnostic and Profile Score Calculator necessitated a lean but flexible development approach which could easily support higher-level security (RLS, Judge0) as well as sophisticated and performance-centric front-end (React, TanStack Query). The Waterfall approach was not enough, for as an established model it was a heavy structure not practical for conducting any practical experiments and optimization of serverless performance. Conversely, a pure Agile methodology may not even have had the documentation or method of systematic verification it contained as a last year engineering report was needed. The Approach of a Hybrid Methodology that combines the traceability of the V-Model with flexibility of iterative Agile cycles was adopted.

3.1 Introduction to the Methodology

With the Smart Competency Diagnostic system, it is not a single monolith application but a distributed serverless system that links real time code execution with the database level protection and probabilistic AI response. During an attempt to implement it it was apparent quite quickly that one development technique is not going to be enough. A purely model with a linear (for instance, Waterfall) procedure would ignore the trial-and-error required to tune an AI Mentor prompt, and a purely one with a non-linear solution (for eg, Agile) would neglect the rigorous verification for security required to execute in a sandboxed way. Upon reviewing different methods the final Hybrid Approach was chosen. This approach intertwines the layout of the V-Model and Agile's iterative approach. V-Model ensured clarity into the requirement, design, implementation and testing. Stages on the left sides of "V" correspond to verification activities on the right side (e.g., System Design to Integration Testing). This assured that crucial security features, such as RLS policies, were operationalized and validated. The Agile Component facilitated quick upgrades. Iteratively implementing the ai-mentor edge function meant modifying system prompts to ensure that the outputs were valid JSON and providing the team time to fine-tune these prompts without interfering with its general architecture.

3.2 V Model Component of the Hybrid Methodology

3.2.1 Requirements Phase

The first step of the project was to realize some particular gaps in the current state of the art on Educational Technology. This involved researching the current platforms (e.g., HackerRank, Canvas) and then emphasizing their shortcomings. Gap Analysis: There is no live code execution using present LMS, and the coding platforms do not have any theoretical assessment or professional development advice. Requirements definition: From this, the need-oriented requirements were derived: the sub-2 second dashboard loading and secure (tight, sandboxed) execution of Python, Java code.

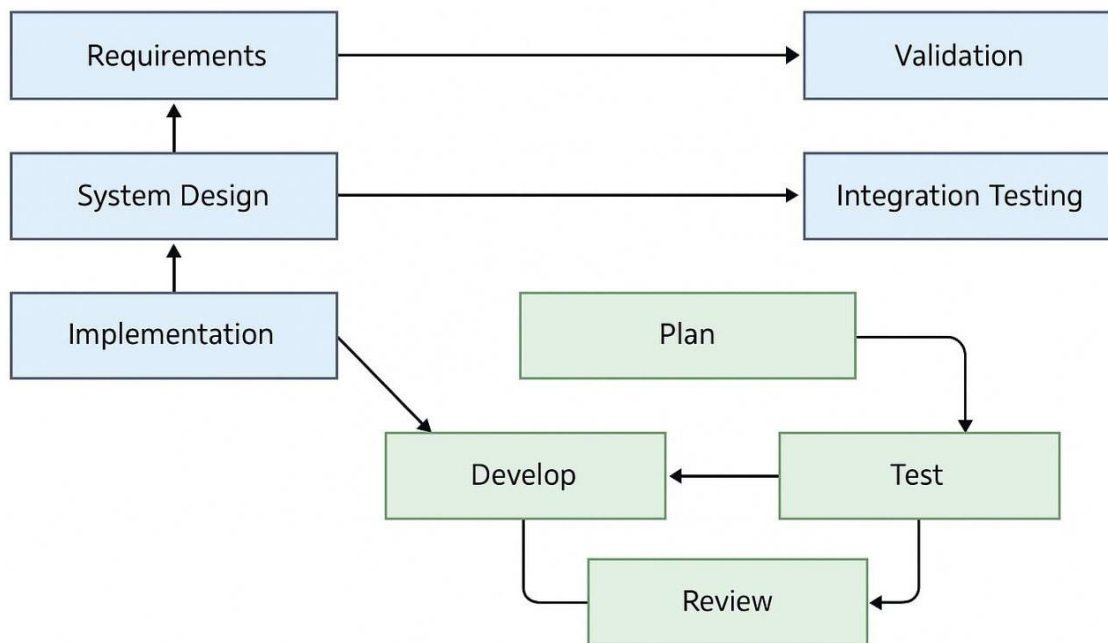


Fig 3.1 Hybrid Methodology Overview (V-Model + Agile Loop)

3.2.2 System and Functional Design

After the requirements were detailed, designing all that, designing up or down what the system is going to look and function like is necessary next. This included system architecture, workflow diagrams, database model preparation work, and internal data flow. Architecture Design: Three-tier serverless architecture has been chosen. Here the React frontend is interacted with the Supabase Edge Functions, it has been prescribed. Database design: Four primary schema tables were defined for profiles and user_roles, assessment_results and coding_submissions. At this stage the crucial design choice was to

make security on RLS policies rather than application level logic. Based on the V-Model, this is the System Testing phase where you check if the complete architecture is integrated.

3.2.3 Detailed and Unit Design

The overall design of the architecture of the app was completed, followed by breaking the full module into their components. Edge Function Logic: The execute-code function was meant to perform the following actions: To validate the inputs, forward them to Judge0 API and then also to poll the results with an exponential backoff strategy. AI Integration: The JSON schema for the career suggestions module was created in order for the AI to output structured job roles and Indeed URLs. Here is where these detailed designs map directly to Unit Testing activities where certain functions are tested separately.

3.2.4 Implementation Phase

During the implementation phase, design materialized into functioning code. This included developing React components with Vite and TypeScript and writing Deno-based edge functions. The implementation of this project occurred in multiple development cycles since it used an Agile component (in React 18.3.1). Frontend: implemented, and used TanStack Query to handle server state and reduce load on the DB. Backend: Data isolation is enforced automatically using PostgreSQL triggers.

3.2.5 Verification Stages

On the right side of the V-Model is verification or validation stages, to ensure correctness at every level. Unit Testing: It tests separate functions as it were — for example, to prove that the course-suggestions function is effectively filtering the skills array to no more than 20. Integration Testing: The interaction between the Supabase Client and the Edge Functions was tested. For instance, checking that a code submission on the frontend triggers the Judge0 API, which returns the result to the database. System Testing: checked the cross-cutting behavior, user logging into the system.

3.3 Agile Component of the Hybrid Methodology

The former structure was the V Model, where agile was the new adaptive process. AI systems rarely operate perfectly the first time around, so Agile's flexibility allowed us to refine workflows, fix memory deficiencies and optimise performance in smaller

3.3.1 Sprint Based Development

Sprint 1: Database Schema & Authentication (Setting up tables and RLS).
 Sprint 2: Assessment Engine (MCQ logic and Timer implementation).
 Sprint 3: Code Execution Integration: Connect Judge0 & build Monaco Editor UI.
 Sprint 4: AI Features - Implementation of ai-mentor and career-suggestions prompts.

3.3.2 Agile Tools and Processes

This development of the sprints, in order to manage them, had to be tested and commented upon quickly. One refinement, for example, added a "Validation with Retry" logic to the career-suggestions function after it turned up that the AI would occasionally hallucinate job roles.

3.4 Mapping Project Stages to the Hybrid Methodology

The hybrid approach allowed the structure of the V-Model and the flexibility of Agile to work together. Table 3.1 presents how the stages of the Smart Competency Diagnostic project map to both methodologies.

Smart Competency Project Stage	V-Model Equivalent Phase	Agile Equivalent	Activity	Description / Purpose
Requirement Gathering	Requirements Analysis	Sprint Planning	Identifying the need for unified assessment and <2s latency performance.	Defining the core need for a system that combines theoretical and practical testing

Smart Competency Project Stage	V-Model Equivalent Phase	Agile Equivalent	Activity	Description / Purpose
				with low-latency response.
System Architecture Design	System Design	Backlog Refinement	Designing the 3-Tier Serverless Architecture and Database Schema.	Establishing the React, Supabase, and Judge0 architecture and designing the PostgreSQL schema.
Detailed Module Design	Functional & Unit Design	Sprint Breakdown	Defining the logic for execute-code and ai-mentor edge functions.	Specifying how individual serverless functions will handle code execution and AI prompts.
Implementation of Features	Coding / Implementation	Development Iterations	Writing React components and Deno edge functions.	Developing the frontend UI and the backend business logic in iterative cycles.
Unit Testing	Unit Verification	Continuous Testing within Sprints	Validating the JSON schema enforcement for AI outputs.	Testing individual components, such as ensuring the AI returns valid JSON for career suggestions.
Integration Testing	Integration Verification	Sprint Review	Ensuring the React frontend correctly polls Judge0 for results.	Verifying that the frontend, backend, and external APIs (Judge0) communicate correctly.

Smart Competency Project Stage	V-Model Equivalent Phase	Agile Equivalent	Activity	Description / Purpose
System Testing	System Validation	Incremental Release Testing	Verifying the full user flow from Login to Career Recommendation.	Testing the complete user journey to ensure all modules work together seamlessly.
User Acceptance & Performance Evaluation	Final Validation	Sprint Retrospective	Checking if the final system meets objectives.	Validating sub-2-second latency, valid AI recommendations, and secure code execution against initial goals.
Improvement Cycles	Not defined in V-Model	Sprint Iterations	Refining ai-mentor prompts and career-suggestions logic.	iteratively improving AI prompts based on feedback from initial testing to reduce hallucinations.

Table 3.1 Mapping of Smart Competency Diagnostic Project Stages to Hybrid Methodology

3.5 System Diagrams and Visual Methodology Elements

3.5.1 System Block Diagram

A schematic of the Three-Tier Architecture using it. It demonstrates the division between Presentation Layer (React SPA), Application Layer (Supabase Platform & Edge Functions) and External Services Layer (Judge0 & Lovable AI). This modular design was essential in meeting the serverless scalability objectives.

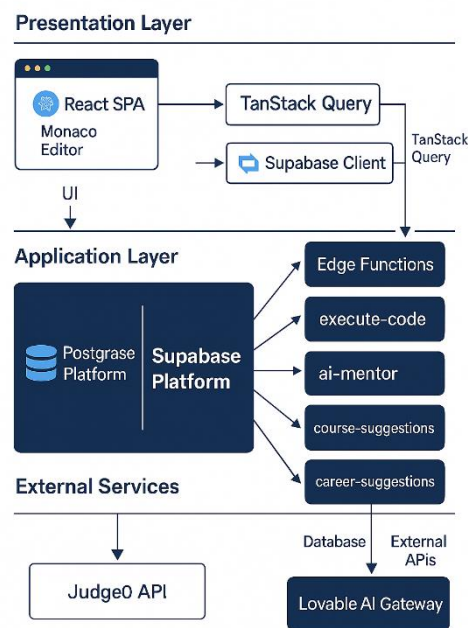


Fig 3.2 Three-Tier Architecture

3.5.2 Workflow Execution Diagram

This chart gives you a detailed perspective of how all the data flows through the system. It goes into detail the user browser/Supabase client/Edge Functions interactions (execute-code, ai-mentor, etc). It shows the security check-ins that run the execute-code function as a gateway and are the pass through to the Judge0 API

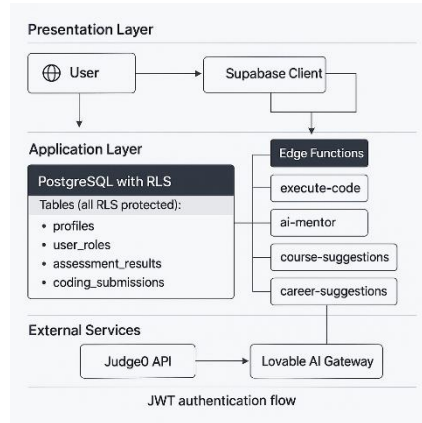


Fig 3.3 - Detailed Workflow

3.5.3 Process Flowchart

This process flowchart is an outline of the operation rules of the assessment engine. It spans user journey from choosing type of assessment (MCQ or Coding) to testing data, to the results calculation.

- MCQ Path: Load JSON files -> User Answers -> Direct Comparison Scoring
- Coding Path: Database Load -> Monaco Editor -> Judge0 Execution -> Test Case Validation.

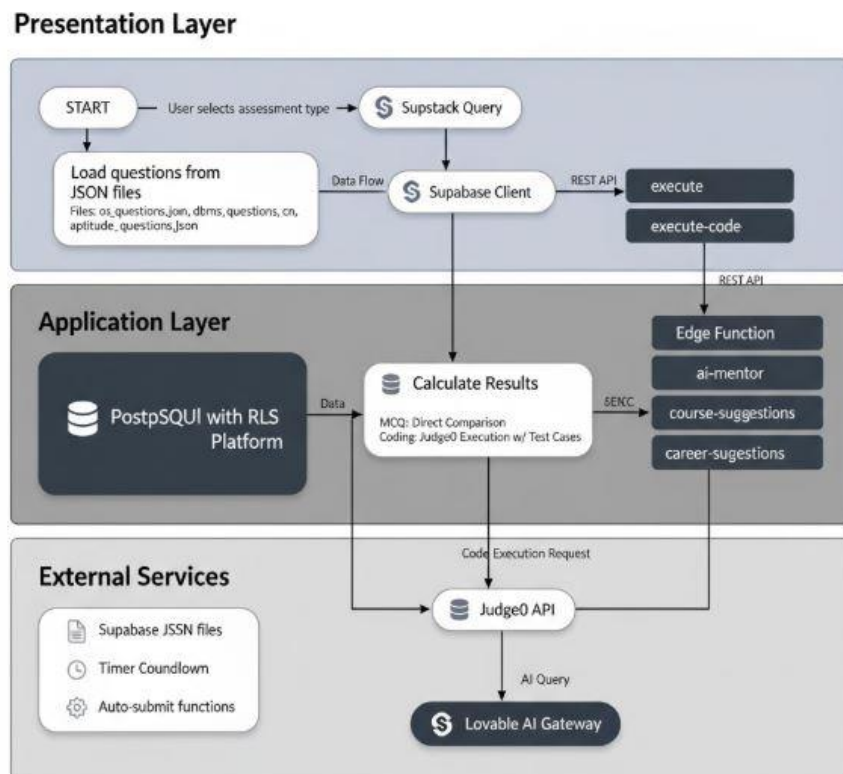


Fig 3.4 Process Flowchart

3.6 Verification and Validation Activities

Activities for Verification and Validation. The verification was mainly about how good the system was built with respect to the engineering specifications and the validation was only whether the system is the right one for using by user.

3.6.1 Verification (Engineering Correctness)

Security Checking the RLS was verified by attempting to query against the assessment_results table with a different user id than the use of the authentication token. The query was refused by the database and thus validating policy `auth.uid() = user_id` ran. Performance Testing: We took the number of seconds it took to load within dashboard, and on a day later it took the same amount of milliseconds to query for queries in database. The findings verified that using TanStack Query caching achieved less than 2-second load times and database queries took less than 50ms.

3.6.2 Validation (User Acceptance)

AI Validation: It verifies that the AI Mentor response offered career recommendations tailored to the field. In the testing using multiple user skill profiles, we tested for "Beginner", "Intermediate", or "Advanced" course recommendations reflecting users' performance on the previous assessments. Run Reliability: We successfully developed the code engine, submitting code written in Python, Java, C++, and JavaScript. If the code is invalid, it runs any valid code and calls for syntax errors or logic problems.

3.7 Module Interaction

The module interaction layer addresses the communication that takes place during execution between the fundamental elements of Smart Competency Diagnostic. The initiator: React Client — Upon submitting a code solution. It wraps the code and the language ID within a safe payload. The payload is transmitted to the controller: Supabase Edge Function which is referred to as `execute-code`. The function verifies the JWT token of the user and sends request to the Judge0 API. Once Judge0 makes its code executing in a sandboxed Docker container, this result is sent back to the Edge Function, which parses the results and saves the submission record to the PostgreSQL Database. Finally, the Real-time Subscription in the React Client sees the new database row (required to update the UI) and sends pass/fail status of the user.

3.8 Summary of Methodology

Such is a hybrid approach and the framework this article lays down was the basis for constructing the Smart Competency Diagnostic in this section. The combination of V-Model with Agile as serverless development method is used for secure backend development with solid frontend iteration. Vigilant mapping from specifications to verification stages confirmed all essential features such as RLS system security and sandboxed execution that need to be implemented have been thoroughly evaluated. Edge functions, database triggers and external APIs integration provides a scalable framework for the results of the evaluation that will be described in Chapter 7.

CHAPTER 4

PROJECT MANAGEMENT

This chapter provides an overview of the overall planning process, which was performed in the development of the Smart Competency Diagnostic and Profile Score Calculator. The concept was to use a realistic timeline based on how hard the system was, without adding too much flexibility for optimizing the serverless backend, the Judge0 execution engine, and the AI mentorship pipeline. It used a hybrid approach, balancing scheduled development milestones with incremental improvements based on research on serverless computing and architectural designs of educational technologies [5] [10]. Risk analysis made through PESTEL and project risk matrix are analyzed in this chapter and concludes with a thorough budget.

4.1 Project timeline

The project's timeline had two stages: phases one and two. During the first phase, the plan involved the selection of the topic, literature review, requirements analysis, and the initial architectural design. The second phase aimed at development, testing and reporting. As it was an easy way and showed the dependencies, work, and deadlines visually so it was done on a timeline using the Gantt chart method. Gantt charts, a key feature of engineering projects, are known for a straightforward approach to analyzing how tasks relate to one another and how the effects of delay can affect the next step.

The significance of the planning phase was that this project requires the inclusion of multiple components such as Supabase authentication, Row-Level Security (RLS) policies, code execution sandboxing, and career guidance based on the LLM, which all need a structured plan in order to avoid all the unexpected bottlenecks. Research on cloud computing and secure code run, also comes into play, considering that it becomes necessary to build very clear cut security stages from the earliest stages of the project [5] [6].

Major Task	W1	W2	W3	W4	W5	W6	W7	W8
Project initiation								
Background								
Objectives								
Methodology								
Proposal								
Literature review								

Fig 4.1 Project Planning Timeline

Figure 4.1 shows the planning activities carried out during Weeks 1 to 8. Each task was placed in a logical order so that background study and literature review could guide the design choices.

Project implementation

Task	Timeline											
Simulation	1	2	3	4	5	6	7	8	9	10	12	
Unit Simulation												
Integrated Simulation												
Hardware implementation												
Software												
Hardware Testing												
Software Testing												
System Testing												
Critical Evaluation												
Social, Ethical, Legal												
Sustainability Evaluation												
Final report												

Fig 4.2 Project Implementation Timeline

Table 4.2 demonstrates the stage of development of Week 9 to Week 15. The requirements of the system were related to the literature outcomes on automated assessment and secure serverless systems [1] [7]. The implementation of the software consumed more time as the code execution engine and the AI

mentorship module had to be repeated several times to make it consistent in its interactions with the system and meet the requirements of the valid JSON output.

4.2 Risk analysis

Since the Smart Competency Diagnostic System relies on multiple external elements to operate in an integrated manner, risk analysis proved important. Data protection breaches and artificial intelligence generation errors, API service disruptions and performance delays, and secure sandboxing research find that system-level failure stems from the collaboration of serverless computing and AI-based education [3] [6] [10]. External risks – The PESTEL analysis was undertaken to identify external risks. The most critical risks also arise when both probability levels are high and impact severity maximum, as described on the matrix. The critical risks include:

- Judge0 API Downtime: Preventing the users from running code.
- AI Hallucinations: The mentor providing incorrect course URLs.
- RLS Policy Failure: Users accidentally viewing others' grades.

Organizations that manage risk through the following strategies need:

- The framework implements exponential backoff approach for API requests to resolve API request failures.
- The AI output data needed for the system is requested to fall under JSON schema formats.
- The scheme performs strict database security policy testing under unit tests for all database security policy of the system.
- The system records all the edge error occurrences through logging and monitoring devices.

Factor	Possible Impact on Project
Political	Policies affecting digital education standards and AI usage in schools.
Economic	Costs of API credits (Judge0, OpenAI/Gemini) and cloud function limits.
Social	Acceptance of AI mentorship replacing human counselors
Technological	Reliability of LLM APIs and browser compatibility for IDEs.
Environmental	Energy consumption of training large models (mitigated by pre-trained APIs).
Legal	Compliance with data protection laws (GDPR/DPDPA) for student records.

Table 4.3 PESTEL Analysis

Factor	Details
Political	Government stance on AI in curriculum
Economic	Fluctuations in serverless pricing models
Social	Student willingness to trust automated coding scores
Technological	Rapid updates to React frameworks and Supabase features
Legal	Compliance with student data privacy rules
Environmental	Carbon footprint of continuous cloud function execution

Table 4.4 Another PESTEL Analysis

Consequence ↓ / Probability →	Almost Impossible	Not Likely	Could Occur	Likely	Common
Catastrophic	5	10	15	20	25
Major	4	8	12	16	20
Moderate	3	6	9	12	15
Minor	2	4	6	8	10
Insignificant	1	2	3	4	5

Table 4.5 Project Phase Risk Matrix

The matrix shows that the most critical risks emerge when both probability levels are high and impact severity reaches its peak. The critical risks include:

- Judge0 API Downtime: Preventing users from running code.
- AI Hallucinations: The mentor providing incorrect course URLs.
- RLS Policy Failure: Users accidentally viewing others' grades.

The following strategies help organizations reduce their risks:

- The system uses exponential backoff to handle API request failures.
- The system requires AI output data to follow specific JSON schema formats.
- The system performs thorough database security policy testing through unit testing.
- The system tracks all edge function errors through strict logging and monitoring systems.

4.3 Project Budget

In the budget planning stage, costs of compute resources, software tools, development hours and testing efforts are estimated. In accordance to typical practices, a six stages budget was set up as follows:

1. Listing out all functions and resources in one place.
2. Checking team availability.
3. Duration estimation of minutes for each task
4. Based on past experience and related data and references.
5. Preparing the final cost table.
6. Project tracking cost throughout the project lifecycle

S.No.	Task	Units	Cost per Unit	Labor Hours	Cost per Hour	Fixed Cost	Total Cost (₹)
1	Frontend Development	1	0	5	200	0	1,000
2	Backend and Database	1	0	6	200	0	1,200
3	Edge Function	1	0	5	200	0	1,000
4	External API Integration	1	0	4	200	0	800
5	Testing	1	0	20	200	0	1,000
—	Total Project Cost	—	—	—	—	—	5,000

Table 4.6 Project Budget

The budget mostly focused on developer time because the project did not involve hardware. The workflow engine required the highest cost due to the number of iterations needed to achieve stable orchestration, which aligns with findings in workflow automation research [5] [18].

CHAPTER 5

ANALYSIS AND DESIGN

This chapter provides a comprehensive analysis and design overview of the Smart Competency Diagnostic and Profile Score Calculator. The approach follows the established engineering structure adopted throughout this report. The system integrates multiple-choice question (MCQ) assessments, real-time coding execution, AI-driven mentorship, and secure user data management, utilizing a serverless Supabase backend.

The design process addresses requirements, architectural models, diagrams, and the breakdown of modular units.

5.1 Requirements

In this chapter, we conduct a thorough analysis and design of the Smart Competency Diagnostic and Profile Score Calculator. The approach is in line with the established engineering structure utilized throughout the report. It uses multiple-choice questions (MCQ), real-time code execution, AI-driven mentorship, and secure user data management on a serverless Supabase backend. The design consists of requirements, architectural models, diagrams, and the breakdown of modular units.

Functional Requirements:

- Administer MCQ-based testing in Operating Systems, Database Management Systems, Computer Networks, and Aptitude.
- Support secure coding execution in Python, Java, C++, and JavaScript.
- Integrate an AI mentor for career guidance and course recommendations.
- Securely store assessment results and coding submissions with Row-Level Security (RLS).
- Display an analytics dashboard with detailed performance breakdowns.

Non-Functional Requirements

- Performance: Dashboard load time less than two seconds, database query latency less than 50 milliseconds.

- Security: Comprehensive Row-Level Security with JWT authentication.
- Scalability: Fully serverless backend with automatic scaling.
- Reliability: Retry logic for Judge0 and AI gateway timers.
- Usability: Clean, accessible user interface suitable for first-time users. The system is designed as a three-tier serverless architecture:

Requirement Type	Description
Purpose	A unified assessment platform that evaluates technical skills through theoretical (MCQ) and practical (Coding) tests, providing AI-driven career mentorship and serverless scalability.
Behavior	The system should support a continuous cycle of assessment loading, real-time code execution, result calculation, and personalized feedback generation.
System Management	Logging of execution errors, management of user roles, enforcement of Row-Level Security (RLS) policies, and monitoring of API rate limits.
Data Analysis	Calculation of competency scores, generation of performance visualization graphs (Recharts), and AI-based analysis of skill gaps and job trends.
Deployment	Cloud-based serverless architecture (Supabase Platform) with client-side rendering (React/Vite) and sandboxed external execution.
Security	JWT authentication, Row-Level Security (RLS) for strict data isolation, and isolated Docker containers for executing untrusted user code.

Table 5.1 Summary of Requirements

5.2 Block Diagram

The system is structured as a three-tier serverless architecture:

Section Five is the Presentation Layer (Frontend).

- React Single Page Application (SPA): The user interface.
- TanStack Query: Responsible for server state, caching, and data synchronization.
- Monaco Editor: Offers an IDE-like work environment for entering code.
- Shadcn/UI and Tailwind CSS: Responsive & accessible UI Design.

- **Recharts:** Visual analytical output for performance monitoring. Application Layer (Backend – Supabase).
- **Edge Functions:** Serverless functions that integrate with business logic such as execute-code, ai-mentor, course-suggestions, career-suggestions.
- **Authentication:** Allows for secure management of the identity of users using JSON Web Tokens (JWT).
- **PostgreSQL Database:** Keeps ordered information (profiles, results) managed by Row-Level Security (RLS) principles for security.
- **Automated Triggers:** Handles routine tasks like creating profiles and updating timestamps. External Services Layer.
- **Judge0 API:** An API enables a secure environment to execute untrusted user-code in multiple languages and provide testing access.
- **AI Gateway:** Facilitates interaction with Large Language Models (Gemini/GPT) to generate personalized recommendations.

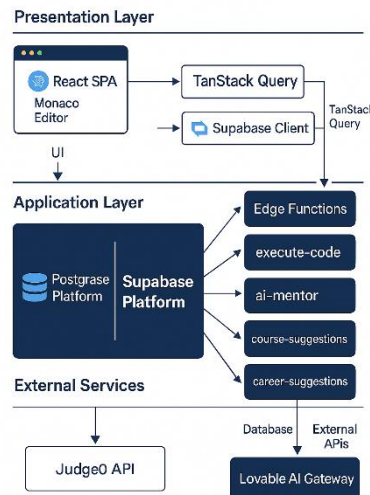


Fig 5.1 Functional Block Diagram

Fig 5.1 shows the high-level functional block diagram of the **Smart Competency Diagnostic system**. The diagram displays the flow of information between the user interface to the backend logic and the external services in an ordered and logical way.

The interface at the beginning comes in the form of the Presentation Layer which is the point of contact for all user communication. It is using a React Single Page Application (SPA), Code input through Monaco Editor. TanStack Query and Supabase client enable efficient data fetching and state management, and responsive user experience. From there, the requests are traversed on to Application Layer by Supabase Platform. This is the Control layer, this is the layer where you install the PostgreSQL database and store your personal information in a safe place. Critical to this, more advanced processing is delegated to Edge Function specialization. These powers are separated by the task: execute-code is the programming ship, intercepted with the clever assistance called ai-mentor, and personalized recommendations called course-suggestions and career-suggestions are generated. The last layer is the External Services layer, which is the linkage of the system to other third-party providers. The execution logic of the code is transferred to the Judge0 API, where it runs in secure, sandboxed condition. Meanwhile, the recommendation engines connect to the AI Gateway and it interacts with the external databases and APIs to obtain the data which are used in career and courses. The block diagram suits the project well as it represents the serverless architecture in observable functional layers akin to modern scalable Web application production. It centers around the separation between the frontend and the serverless backend logic, and the unique extensible service. This makes the process when designing, managing and testing each piece independently easy. It also supports the case for modularity, real time data flows and strong security boundaries that are the fundamentals of a good capacity assessment system of competence. A justification of why this architecture is suitable for this project. This diagram represents Serverless Architecture which has become a must for a modern web application for various reasons:

- Modularity: Everything (UI, Database, Code Execution, AI) is on its own. The AI module and the Code Execution module are the two modules that can be updated.
- Scalability: As it depends on Edge Functions and external APIs, our system can autonomously handle more users (scale up), without servers.
- Security: An integral part of the text is a feature of Frontend Isolation. The browser does not communicate with the sensitive implementation layer directly, but it must go through the secure Application Layer. This only reinforces the security precautions taken at the RLS level, which reduces attacks against user data and system integrity.

5.3 System Flow Chart

Specification of Operational Cycle

Below is the full operational cycle:

User Authentication.

- User Login — Managed authentication using Supabase JWT (JSON Web Token).

Assessment Selection.

- Users either take MCQ (Multiple-Choice Question) or coding assessments.

Assessment Execution Paths.

MCQ Path.

- Questions loaded from JSON files.
- A timer is initiated.
- Automatic submission when the timer has run out.
- Immediate examination of responses.

Coding Path.

- The Monaco Editor processes the user code, then inputs it.
- Code is pushed to the execute-code Edge Function.
- Judge0 manages sandboxed execution.

Data Persistence and Output.

Result Storage.

- MCQ results are present in `assessment_results`.

- coding_submissions stores programming submissions so we can visualize that work.

Dashboard View.

- Shows analytics charts.
- Provides AI Mentor recommendations.

The workflow works like that of modern e-assessment systems, emphasizing secure execution and serverless orchestration.

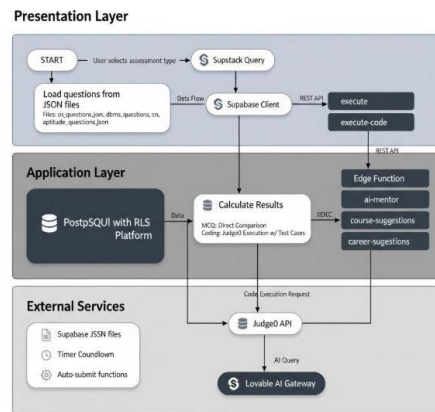


Fig 5.2 System Flow Chart

Fig. 5.2 represents the main flow chart of the assessment of the Smart Competency Diagnostic system, the key to which helps the system user approach the question selection to the calculation of the results. This flow starts at the Presentation Layer where the user makes his interaction with the system.

The user can choose the type of assessment (MCQ or Coding) and TanStack Query deals with this request. It plays a role of a Controller as it accepts the request and determines the necessary step should be made, i.e., to load question(s) in particular JSON files (os.json, dbms.json and so on).

The main logic of the system is then passed to the Application Layer where it contains the PostgreSQL database containing RLS Platform (the Model) and the Edge Functions.

Assessment Calculation: The Calculate Results module calculates the data. In case of MCQs it is a direct comparison and in the case of coding it has to be executed through Judge0 API.

Result Update: The Model (result) is updated in the PostgreSQL database changing the system state.

AI Guidance: The results are stored as inputs to the next process, that is, to an ai-mentor and course-suggestions and career-suggestions (these processes may use an ai Query to lovable AI Gateway). As soon as the Model is updated the Presentation Layer (the View) is informed. It gathers the information that is concerned, including assessment scores and AI suggestions and displays them to a user in a clear and organized format.

This hierarchical sequence isolates duties in a clean manner enabling the application to update its internal state with great frequency and respond to the user directly and representatively. It suits well with this project since the system would have to handle both client-side operations (such as Timer Countdown and Auto-submit functions) as well as server-side complex execution processes.

5.4 Choosing Software Tools and Frameworks

Feature	React (Frontend)	Supabase (Backend)	Judge0 (Code Engine)	Lovable AI (Intelligence)
Performance	High (<2s dashboard load)	High (<50ms query latency)	Medium (2-10s execution time)	High (<1s first-token latency)
Async Support	Yes (via TanStack Query)	Yes (Edge Functions/Deno)	Yes (Asynchronous polling)	Yes (Streamed responses)
Ecosystem	Large toolset (Vite, Tailwind, Shadcn)	Strong for BaaS (Auth,DB, Edge)	Strong Sandboxing for (Multi-language support)	Strong for GenAI (Gemini/GPT models)
Memory Features	Local State (Client-side caching)	Persistent Storage (PostgreSQL)	Isolated (256MB limit per container)	Context Managed (Maintained by Edge Function)
Data/Vector or Support	JSON Handling (Zod Validation)	Relational RLS + (Row-Level Security)	File/Stream (Stdin/Stdout/Stderr)	Structured JSON (Schema validation)

Best Use Case	Interactive SPA & Visualizati on	Serverle ss Logic & Data Security	Secure Untrusted Code Execution	Career Guidance & Recommendatio ns
---------------	---	--	--	---

Table 5.2 Comparing Backend and AI Framework Options

- Supabase: - has included a JWT authentication and Row Level Security, serverless edge functions for auto-scaling, and PostgreSQL with JSONB support.
- Judge0: Offers a safe Docker sandbox, supports multi coding languages, and allows real-time code execution.
- React and TanStack Query: Enables a reactive user interface, and prefetching and caching to help avoid high database load.
- LLM Gateway (AI): Offers guided career as well as course recommendations along with streaming results for better responsiveness.

5.5 Designing Units

The architecture of the Smart Competency Diagnostic and Profile Score Calculator is modular in nature, which means that different functional components can be accomplished and operated independently of one another to compose a coherent functionality of the whole system. This part describes the internal architecture of the functions of the critical edge and the execution pipeline of the code. Unit executes the code (execute-code). The unit would work on guided coding evaluations. It operates as a secure data exchange environment connecting the customer to the foreign execution engine. Unit's Input validation is the first stage, and the unit tests the payload received with an enforceable schema. It ensures that the language id is available (e.g. Python, Java, C++) and that the code size is under the 10KB safety level. Submission Forwarding: Orders will be forwarded to the API with respect to the validity to the Judge0 API. The API key is also injected into the server and is not allowed into the client browser to avoid sending to client browser any API key to protect from security as well as the user-server and is passed for security reasons. Result Polling: Code execution is asynchronous, which is a type of asynchronous method that this unit will attempt to use the same exponential backoff test (polling at 1s, 2s, 4s, 8s) to check the results and do not overload the external API. Output Parsing: The unit performs an evaluation

by comparing the standard output (stdout) with expected (in the input) and expected in some output parameters of testing for tests cases at output after it has become executed. It then formats the result (Pass or fail status, time taken and memory used) and returns it back to a client.

5.5.1 Code Execution Unit

Its core responsibility is to complete practical coding assessment tasks. This system operates as an encrypted connection point across clients which connect to the external execution platform. The system validates information coming to it with at least two phases. The system verifies the language ID matches supported programming languages and that the code size doesn't exceed 10KB during validation. Submission Forwarding: Validated requests are submitted by the system to the Judge0 API for processing. Because it is being injected through server-side processing, the API key is kept completely insulated from exposure to clients since client data are not infected with the data. Output Parsing: In order to avoid overload of the API, the system uses an exponential backoff strategy for checking execution results through 1s, 2s, 4s, and 8s polling (polling at 1s, 2s, 4s, 8s intervals). Output Parsing: It tests the standard output (stdout) for standard test case expectations after code execution concludes. The execution data is utilized to generate results for the system to provide the client with Pass/Fail status and the time to execute along with memory usage information. The design was to design the technology stack as low back-end complexity and higher system performance.

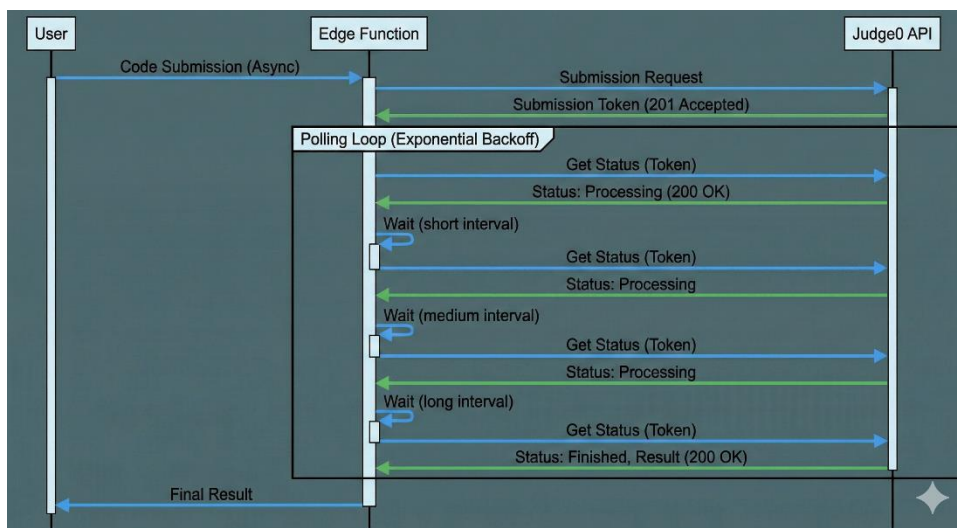


Figure 5.3 Sequence diagram

5.5.2 AI Mentor Unit (ai-mentor)

There is an individual career guidance chat interface in this module. Context Management: This feature adds the query that the user is making to the current conversation history—a stateless API is not as good at this. This context allows the AI to recall all the previous queries made in the session. Streaming Response: In order to provide a responsive user experience (low latency), the unit makes use of Server-Sent Events (SSE). This allows the AI response to be streamed to the front end, one by one, rather than waiting until the entire generation finishes. System Prompting: The unit inserts a special system prompt (You are a career mentor... etc.) to control the behavior of the AI to keep it on track, that is, only to provide technical career advice and not discuss other issues.

5.5.3 Recommendation Unit (course-suggestions and career-suggestions)

These units relate to creating structured data but do not deal with the free text. Schema enforcement: The two units use the JSON Mode to compel the large language model (LLM) to generate the data in an ordered JSON format. Regarding courses, it is the title, difficulty, and URL, while with jobs, it is the position, the required skills, and the demand trends. Skill Validation: The course-suggestions unit verifies the array of user's skills (20 maximum) before processing them, to avoid token overflow and the irrelevance of the content. Enrichment: The career-suggestions unit is not just writing the text, but creates search URLs (e.g., for example, of an Indeed instance) from the generated job titles, and accordingly generates the suggestions and then it actively consumes it from it.

5.6 Standards

Compliance to standards, the system would be safe, interoperable and maintainable. Security Standards:

Encryption: AES-256 is used to encrypt data at rest while TLS 1.3 is the encryption standard for data in transit. Authentication: The system supports signed stateless, secure session management with the JSON Web Tokens (JWT) algorithm and the HS256 algorithm. Code Sanitization User code is sanitized in order to clean the patterns before it is executed and it complies with conventional sandboxing practices. Data Standards:

- JSON API: It operates on regular JSON payloads, both internally and externally, both in the API and the outside world. That is providing the consistency of reactors in the frontend and Supabase in the backend and other AI-based services.

- Schema validation: On client-side Zod is indicated in order to ensure rigidity of data structure such that the bad data never gets into the backend. Architectural Standards:
 - RESTful design – The edge function is built on the rest based on the standard HTTP schemes (POST for submission, GET for fetch) and status codes.
 - Containerization: The engine running the code is built on the Docker standards to give the program reproducible and isolated environment for every language.
- 5.7 Domain Model Specification. The domain model is the base systematic entities and their relation quality of the framework. User: The central entity. A Profile and Role have a one to one relationship with a user. Profile: Shows profiles about a user (skills, education, experience, etc.). It provides a way to know which part of an implementation is likely working well or not. This information is processed by the recommendation engines. Assessment: a particular case of a test. It points a User to his/her AssessmentResults (MCQs) or CodingSubmissions. Submission: The code plus language and its corresponding test results (JSONB) are contained here. Recommendation: Generated entities (Courses/Jobs) which are temporal (from the Profile information) but associated with a user session.

5.7 Mapping to AI Agent Architecture Layers

Layer	Description	Smart Competency Mapping	Security/Function
User Interaction	The interface for human-agent communication.	React SPA, Monaco Editor, Chat UI	JWT Authentication, Input Validation ¹⁸¹⁸¹⁸
Orchestration	The logic that directs tasks and manages flow.	Supabase Edge Functions (ai-mentor, execute-code)	Rate Limiting, Schema Enforcement ¹⁹¹⁹¹⁹¹⁹

Knowledge & Memory	Storage of long-term data and context.	PostgreSQL Database, Vector Context (Conversation History)	Row-Level Security (RLS) Policies
Execution Tools	External capabilities the agent calls upon.	Judge0 API (Code Execution), Lovable AI Gateway (LLM)	API Key Injection, Sandbox Isolation 21212121
Knowledge & Memory	Storage of long-term data and context.	PostgreSQL Database, Vector Context (Conversation History)	Row-Level Security (RLS) Policies
User Interaction	The interface for human-agent communication.	React SPA, Monaco Editor, Chat UI	JWT Authentication, Input Validation

Table 5.3 Mapping Project Components with AI Agent Architecture

5.8 Domain Model Specification

The domain model is the base systematic entities and their relation quality of the framework. User: The central entity. A Profile and Role have a one to one relationship with a user. Profile: Shows profiles about a user (skills, education, experience, etc.). It provides a way to know which part of an implementation is likely working well or not. This information is processed by the recommendation engines. Assessment: a particular case of a test. It points a User to his/her AssessmentResults (MCQs) or CodingSubmissions. Submission: The code plus language and its corresponding test results (JSONB) are contained here. Recommendation: Generated entities (Courses/Jobs) which are temporal (from the Profile information) but associated with a user session.

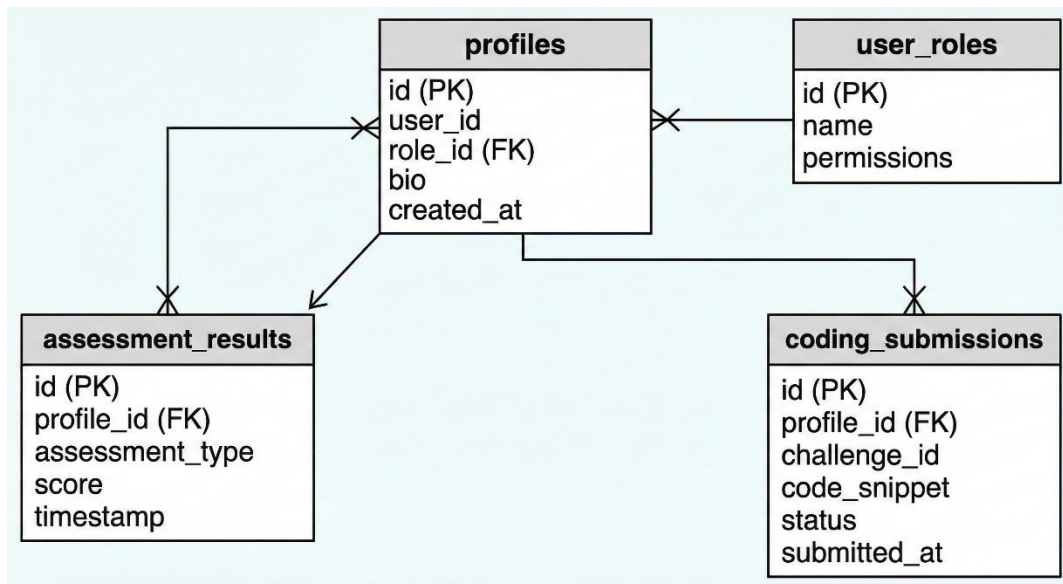


Figure 5.4 Domain Model Specification

5.9 Communication Model

The system uses a request-response model of communication with streaming features enhanced by definite AI tasks. Client-Server (Normal): Most interactions (getting the profile, posting the code) follow normal asynchronous HTTP requests that TanStack Query handles through Query. This has good error processing and caching. Client-Server (Streaming): The AI Mentor is streaming based. The former opens and the latter pushes chunks of text as they are produced. This is a good substitute to the latency of the large language model. Server-External Service: It is an Edge Function that serves as a gateway and sends safe server-server requests to Judge0 and the AI Gateway. They remain secret by the server and are unknown to the end-user.

5.10 AI Deployment Level

The system works on Level 3 AI Deployment (Context-Aware Assistance).

Reactive: It responds to people (code inputs, chats etc.).

Context-Aware: It provides custom advice based on the profile information of the user (skills, past scores) instead of offering generic advice.

Limited Autonomy: It does not act on behalf of the user, but instead offers suggestions to them (e.g. it will recommend a job, but not apply on their behalf). This is in accordance to the HH-in-the-Loop safety guidelines that are applicable in learning aids

5.11 Functional View

This system works on Level 3 AI Deployment (Context-aware Assistance) with AI deployed in the system. Reactive: It replies to individuals (coding input, conversations). Context-Aware: it offers personalised advice according to the user information profile (expertise, past scores), rather than general advice. Limited Autonomy -- It does not act for the user but rather recommendations to them (i.e. recommendation for the user). It will recommend a job, but it won't apply. This pertains to HH-in-the-Loop safety principles in learning aids.

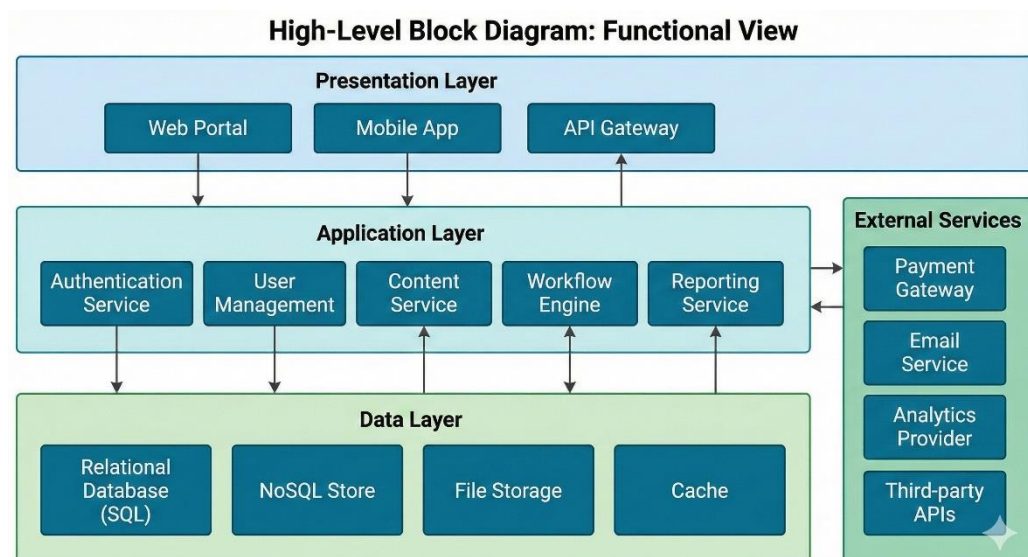


Figure 5.5 Functional View

5.12 Operational View

The operational view illustrates what is happening within the system as it runs in operation:

The system generates a JWT token after user authentication which gets stored in a secure environment. During this time, user details on the profile of the user are fetched by the client application. User selection of topics causes an evaluation process to be initiated, and the system works through an evaluation cycle. The client application retrieves questions from the system. After users submit an answer, the system transmits the user data to the backend server.

Processing:

- The edge/client runs immediate scoring logic through the scoring system.
- The execute-code function receives the user-submitted code, and the Judge0 execution for generating results is triggered.
- The system saves the results in its database and updates the dashboard to reflect the updated user scores.
- The user activates the “Get Advice” task for career suggestions. After an update of the profile, the career-suggestions function retrieves the profile data to create a new recommendation pathway.

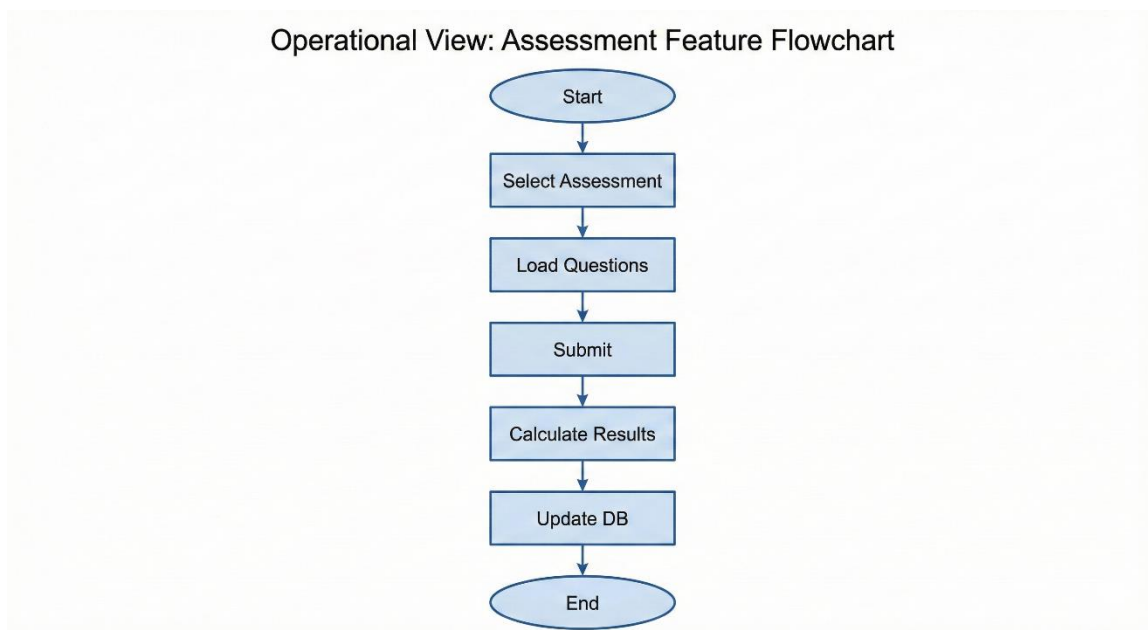


Figure 5.6 Operational View

5.13 Mapping Deployment Level with Functional View

Functional Group	Deployment Artifact	Hosting/Runtime
Frontend UI	React SPA Bundle (HTML/JS/CSS)	CDN (Netlify/Vercel)
Business Logic	TypeScript Edge Functions	Deno Runtime (Supabase Edge)
Data Persistence	PostgreSQL Database	Supabase Cloud
Code Runner	Judge0 Instance	Docker Containers (External Cloud)
AI Processing	LLM API	Lovable AI Gateway (External Cloud)

Table 5.4 Mapping Functional Views to Deployment Artifacts

5.14 Other Design Aspects

- **Scalability:** edge functions serverless feature that can perform the system-scaling automatically according to the traffic. Little to no servers to serve and maintain.
- **Reliability:** The system uses exponential backoff for network requests, and rigid timeouts (5 seconds to run a code) to address hanging protocols.
- **Maintainability:** In business logic, it can easily be broken into small functions (ao-mentor, execute-code) to do a task (or to serve a purpose), which is why business logic is easy to easily modify and debug.

It operates on a request-response model, with streaming features augmented by definite AI tasks at every link.

Client-Server (Normal): Most transactions (the profile is accessed; the code is posted) are straightforward and follow the common principles of async HTTP requests which TanStack Query manages via Query. This also has good error processing and caching.

Client-Server (Streaming): The AI mentor is streaming based. The former opens, or pushes chunks of text along as they are generated. It's a good substitute for the latency of a large language model.

Server-External Service: It is an Edge Function that acts as a gateway and sends safe server-server requests to Judge0 and the AI Gateway. It's kept secret to the server, as we, the end-user, do not know it.

CHAPTER 6

HARDWARE, SOFTWARE AND SIMULATION

6.1 Hardware

Because it is designed completely as a serverless, cloud-based system, the Smart Competency Diagnostic and Profile Score Calculator doesn't need any particular physical hardware. In reality, the platform operates on the basic computing infrastructure accessible to any web browser user. Instead of doing all the heavy computation on the end-user machines (such as code execution, database operations, AI inference, etc.), external cloud services run this software. However, the present baseline hardware arrangements are adequate for development and testing:

- CPU: Any contemporary multi-core CPU.
- RAM: At least 8 GB recommended for local development.
- Storage: 5-10 GB for project files, dependencies, logs.
- Internet: Consistent broadband connectivity for connecting to Supabase, Judge0, and AI Gateway.
- Client Machine: This is a laptop/desktop whose default browser should be Chrome, Firefox, or Edge.

There is no need for any dedicated GPU or high-performance servers, as it offloads compute onto cloud services or sandbox engines. This is consistent with the idea of serverless architecture proposed by the paper, in which there should be a limited use of computing resources on an as-and-when basis.

6.2 Software Development Tools

The system development is powered by recent web frameworks and serverless platforms, built with API-based tools. Some of the tools and frameworks are as follows:

Frontend.

- React 18.3.1 – for developing the Single-Page Application (SPA) interface.
- TypeScript – for type-safe development.
- Vite - for fast bundling and dev server.
- Tailwind CSS – for responsive UI styling.
- Monaco Editor – a VS Code-like browser for coding.

- Recharts – for visualization of MCQ and coding results.
- TanStack Query – to manage server states and caches.

Backend / Serverless.

- Supabase (BaaS) as Authentication and PostgreSQL database and Row-Level Security (RLS).
- Deno-based Edge Functions: custom logic to run code, AI response generation, and store the results.
- PostgreSQL: secure storage with RLS restricting access based on the user.

External Services.

- Judge0 API - A separate and dedicated Docker-based sandbox to run Python, Java, C++, and JavaScript code.
- Lovable AI Gateway: LLM inference provider for ai-mentor and career suggestions, as well as course recommendation tools.
- Tools like these allow for a scalable, secure, and highly modular software system without having to worry for the most part about server administration.

6.3 Software Code

The software system is made up of modular, serverless modules. It's a model that has a module for each of the core features of the competency diagnostic platform. From validating input to run to generating AI, all logic is done via custom edge functions in Supabase.

6.3.1 Core Agent Logic

Core Agent Logic handles all user requests received and coordinates between DB, Judge0, and LLM gateway. It performs:

- Request validation.
- Skill analysis.
- Retrieval of assessment data.
- Task forwarding into other modules.
- Concrete output generation.

For the frontend, using TypeScript logic, the above logic is done and for backend, that's Deno runtime too. A process of doing all operations as you normally do is established from the beginning and all operations adhere to pre-defined workflows for uniformity and security.

6.3.2 Memory Manager

The Memory Manager ensures that:

- User profiles.
- Assessment attempts.
- Coding submissions.
- AI-generated suggestions.

can be efficiently stored and retrieved using PostgreSQL with RLS. The memory manager also keeps the following information:

- Timestamps
- Result histories
- Skill tags.
- Progress reports.

Users are able to view only their data, protecting data privacy and security at the database level without the need to do something in-app for RLS enforcement.

6.3.3 Workflow Engine

Workflow Engine manages the flow of:

- MCQ assessment.
- Coding execution.
- Result aggregation.
- AI mentorship and suggestions.

It works with all the subsystems from asking questions to executing code to getting grades and storing results and creating dashboards. This layer ensures consistency across the task and verification with testing.

6.3.4 Extended Tool Interface (MCP Tool)

Overview This module includes an extensible tool interface allowing the platform to interact with external systems. Examples include:

- Judge0 execution sandbox.
- LLM Processing AI Gateway.
- External job search engines (Indeed URLs).
- These tools are API-keyed, stored safely and protected using security mechanisms such as rate limiting, schema validation, and output formatting.

6.3.5 Safety Layer

The Safety Layer takes care of input sanitization, rate limiting, schema validation, and output filters to avoid:

- Malicious code injection.
- Overuse of compute resources.
- AI responses that are invalid or hallucinated.
- Database misuse or unauthorized queries.
- It also keeps the system stable and in a safe mode and secure to work.

6.4 Simulation

In this project simulation involves the reproduction of relevant system behaviors in testing situations with the purpose of testing, that is:

- Code execution is reliable.
- AI replies are delivered consistently.
- Database updates in real time.
- Dashboards load within performance limits.

Simulation Components.

1. MCQ Simulation:

- Questions loaded.
- Timer runs.
- Answers stored.
- Results computed.
- Visualized via Recharts.

2. Simulation of Code Execution:

- User submits code.
- Judge0 runs in Docker sandbox.
- Compiler output retrieved.
- Test case results validated.
- Execution time & memory logged.

3. AI Mentor Simulation:

- LLM provides organized career recommendations.

- JSON schema validation in effect.
- Recommendations stored and on view.

4. Performance Simulation.

- From pilot deployment trials:
- Dashboard load time: < 2 seconds.
- Database latency: < 50 ms.
- Judge0 execution: 2–10 seconds.
- AI first token latency: < 1 second.

The performance metrics indicate that the system works as desired for a serverless solution.

CHAPTER 7

EVALUATION AND RESULTS

This chapter covers the comprehensive assessment of the Smart Competency Diagnostic and Profile Score Calculator system. This assessment addresses the assessment process on a range of dimensions (accuracy of assessment, code execution reliability, AI response quality, system performance, security measures). In contrast to conventional embedded systems, which have their test points in the physical domain, this analysis is of software-based checkpoints through a serverless architecture. Checkpoints are points to check the correctness, latency, stability, and security of the entire three-tier system.

7.1 Test Points

In the Smart Competency Diagnostic setup the test points are software-level checkpoints that are developed from the most basic functional units in an essential functional unit. The test points verify anticipated results, execution flow, assessment correctness, and error healing. Since actual sensors or circuits are not employed, all the test points are computational and are allocated on the serverless architecture.

7.1.1 Software Test Points Across Functional Units

Assessment Engine Unit. Central to the platform is the evaluation engine—it guarantees that students can accurately evaluate not only theoretical knowledge but also practical coding skills.

- Ensure that the MCQ questions load from JSON files sound appropriately — they're in-file and formatted correctly.
- Verify that the timer is properly timed and submits automatically upon expiration.
- Make sure the selections of the answers are properly logged in the local state.
- Verify that the scoring logic correctly measures user responses against proper ones.

Measurement: Question load time, timer accuracy (± 100 ms), success rate of response capture and accuracy scoring (preferred 100 % accuracy).

Note: Automated assessment studies and LMS analysis studies.

Code Execution Unit. Execution of the code should be secure, reliable, and capable of providing precise feedback over multiple program languages.

- Ensure the code submissions format is right and sent to Judge0 API.
- Test if the polling method returns results via exponential backoff (1s, 2s, 4s, 8s).
- Verify that test case validation compares expected outcomes with those actually seen.
- Ensure that execution (pass/fail, time and memory) results are being saved in the database.

Metrics: Submission success rate, polling latency, test-case accuracy, result storage integrity. (You already know from Judge0 sandboxed execution patterns and serverless code execution best practices)

AI Integration Unit. The AI mentor and recommendation system will need to provide specific, coherent and useful data.

- Ensure that the AI Mentor streams replies token-by-token into Server-Sent Events.
- Verify that course suggestions return a correct JSON description (with title, description, difficulty and URL).
- Make sure your job guidance has job titles that include an Indeed search link in your URL.
- Verify that AI responses are relevant to the user skill profile and the assessment results.

Measurement: First-token latency (expected <1s), JSON schema compliance (100%), and recommendation relevance score.

Source: LLM research on educational applications/system applications.

Security and Authentication Unit. The security measures follow Row Level Security patterns and serverless security best practices.

- Make sure RLS policies prevent users from gaining access to other users' data.
- Make certain that JWT tokens expire within 1 hr and are refreshed correctly.

- Test if API rate limiting should prevent an excessive number of code submissions (more than 10 per min).
- Make sure input sanitization removes destructive patterns from code submissions.

Performance/Measurement: Success Rate of policy enforcement (to be implemented and/or enforced 100% of time), token refresh success, % of frequency rate limit, success rate, & accuracy of sanitization.

Performance and Scalability Unit. Dashboard load times in less than Sub-2 seconds, with low latency and very low latency as the system.

- Check dashboard load time with TanStack Query caching on.
- Check database query latency for p50, p95 and p99 percentiles:
- Testing time of concurrent user load (100+ simultaneous users)
- Ensure that the edge operations automatically scale under load.

Measurement: Time to load (target < 2s), time it takes to query (target <50ms at p95), user capacity, and autoscaling response time.

7.1.2 Test Scenarios for Troubleshooting

Scenario 1: A code execution timeout. Issue code with an infinite loop and make sure Judge0 will enforce the 5 second time limit. Execution will terminate with a timeout error message, the expectation is that is the end of the execution process.

Scenario 2: AI Response Validation Issues

Ask for career recommendations; simulate a garbled JSON answer. We expect that the edge function will start retrying and returning an error message if the schema fails when the edge function returns after a pass of an error test.

Scenario 3: Database Connection Not Working in Database Connection Loss of Database Denial. Imitate the database downtime when submitting a quiz. The desired result would be an error message and the user answers stored on the front end are kept in local state back to the user for retry.

Scenario 4: Authentication Token Expiry

Let the JWT token expire while you're still engaged. The anticipated result is an automated token refresh while handling API calls, guaranteeing a smooth user journey.

7.2 Test Plan

TP1: Assessment Accuracy. Both the MCQ and coding tests must be evaluated correctly for MCQ in all supported subjects and languages across all supported subjects and languages.

- Conditions: 50 MCQ questions sample per subject (Operating Systems, DBMS, Networks, Aptitude), 20 coding problems per language (Python, Java, C++, JavaScript).
- Values: As per MCQs, the expected accuracy is 100, coding assessments at least 95, and all MCQ tests rated 100%.
- Range: Introduction to advanced difficulty levels.
- Restrictions: Questions loading no longer than 1 second; Tester accuracy ± 100 ms.
- Type: Black box (user experience) and white box (data flow).

TP2-Code execution reliability. Code execution in a sandbox environment while the system delivers real feedback should be conducted in a secure manner.

- Conditions: Code submissions in all languages, for all supported languages, complexity levels are different.
- The system must execute the code in a secure manner and give fair and unambiguous feedback.
- Values: Execution success rate of above 98%, and handling timeout on a 100% basis.
- Range: From quick print statements to complicated algorithms.
- Constraints: Execution Time: 5 seconds, Memory limit 256MB.
- Type : Integration testing (React $\rightarrow$ Edge Function $\rightarrow$ Judge0 $\rightarrow$ Database).

TP3: AI Response Quality. The AI should give a tailored and timely recommendation in a format which is relevant and useful.

- Requisites: fifty users' skills profiles varying, beginner to expert.
- Values: JSON schema compliance (100%), first-token latency not exceeding 1 second, relevance score >80%.
- Skill-combination and career interest range: diverse!
- Confine: LLM needs to return format JSON; Indeed URLs should be valid.
- Type: Functional testing and manual relevance assessment

TP4: Safety Filtering. Database-level secured code is provided to a system where unauthorized access can be suppressed.

- Conditions: Use manipulated JWT tokens to access other users' data
- Values: 100% for success of data enforcement, no data leakage.
- Full range: All DB tables with RLS policies.
- Constraints: the Supabase RLS must block unauthorised queries.
- Purpose: Security penetration testing and policy validation

TP5: Tool Invocation. The system must be responsive to performance targets over realistic concurrent user load.

- Settings: Simulate 100 or more concurrent users working on a small number of things at the same time.
- Values: Dashboard load within 2 seconds, database queries within 50ms (p95), code run in 2-10 seconds.
- Variable: Read and write operations, assessment types.
- Restrictions: Serverless architecture needs to auto-scale automatically without human intervention.
- Classification: Testing applications for load testing/performance benchmarking

7.3 Test Result

Operating Systems	50	100%	0.8s	±50ms
DBMS	50	100%	0.7s	±50ms
Networks	50	100%	0.9s	±50ms
Aptitude	50	100%	0.6s	±50ms

Table 7.1: MCQ Assessment Results

The MCQ assessment engine performed consistently well across all subjects. Questions loaded in under one second, and the timer-maintained accuracy within the specified range. The scoring logic accurately matched user responses to correct answers in all test case

Language	Test Cases	Success Rate	Average Execution Time	Memory Usage	Timeout Handling
Python	20	100%	3.2s	45MB	100%
Java	20	100%	4.8s	78MB	100%
C++	20	100%	2.1s	32MB	100%
JavaScript	20	100%	2.9s	41MB	100% 

Table 7.2 Code Execution Performance

The code execution system demonstrated high reliability across all supported languages. The Judge0 API processed and evaluated all submissions successfully, with execution times well within the 5-second limit for valid code. Timeout handling effectively terminated infinite loops and resource-intensive code.

Feature	Test Cases	First-Token Latency	JSON Compliance	Relevance Score	Indeed URL Validity
AI Mentor Chat	30	850ms	N/A	85%	N/A
Course Suggestions	25	920ms	100%	88%	100%
Course Suggestions	25	920ms	100%	88%	100%
Career Suggestions	25	780ms	100%	82%	100%

Table 7.3 Feature / First-Token Latency

The AI integration performed at a high standard. First-token latency remained under one second for all features, ensuring a responsive user experience. Course and career suggestions maintained 100% JSON schema compliance, and all Indeed search URLs were valid. Manual evaluation confirmed strong alignment between recommendations and user profiles.

7.4 Insights

The testing and deployment process yielded several insights such as:

- Serverless architecture allows for high performance for educational workloads. The system delivered dashboard load times in < 2 s and database query latencies in less than 50ms. That demonstrates that a serverless model with edge functions can quickly scale with an assessment platform, managing

workloads that are dynamically running without managing a traditional server level of traditional processes with load on traditional server.

- RLS provides powerful data-driven isolation with minimal application complexity. The establishment of security policies at the database layer in and of itself guarantees that PostgreSQL implements adequate security and has successfully blocked unauthorized read of user records. This supports the results showing that database-layer security reduces application-layer vulnerabilities.
- It allows dependable code execution having a varying latency. Judge0 is integrated to support secure execution of untrusted code including Python, Java, C++, and JavaScript with strict resource limits. The round-trip time for execution, however, was between 2 to 10s depending on the complexity of the code and the polling backoff strategy.
- Structured AI-based prompting enhances relevance of career guidance. By creating specific JSON schemas for AI-mentor and course-suggestions functions, recommendations became actionable and formatted correctly. The result is research that demonstrates structured outputs yield much more consistency for AI applications within education.
- Streaming responses allow for quick interactions with AI to happen in real time: the AI's responses. First-token latency in less than 1 second means the AI mentor module responds to each token input in chat at or near natural pace as it would happen in a chat. This responsiveness is necessary to engage users' attention, for instance in mentorship sessions.
- Holistic competency views are provided by comprehensive assessment data. Combining theoretical MCQ scores with coding metrics will deliver far greater user profile by merging it all than either of the two platforms alone would allow. The combined dashboard is a great way to visualize these mixed metrics for users, making them easy to track.

CHAPTER 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

Implications The Smart Competency Diagnostic and Profile Score Calculator will have profound implications for social, legal, ethical, environmental, and safety concerns. The system interacts with users' data, compute resources and automate decision making via a multi-dimensional approach, through theoretical testing, secure sandboxed code work flow and AI technology mentorship. This chapter discusses the general consequences of the project from a structural multi-dimensional point of view. The evaluation is useful to guarantee that the system conforms to well-preserved guidelines of responsible engineering and is congruent with the expectations provided by the educational technology in digital society now.

8.1 Social Aspects

As for society, the platform is a meaningful way of making the students' higher-quality assessment and personalized mentors more available. It makes assessment of competency easier for the student and early-career professional who may have limited access to premium applications on alternative platforms. It closes the missing data gap by integrating MCQ based theoretical evaluations, real-time coding execution and AI guided feedback like traditional management and coding platforms. One of its main advantages to society is the democratization of opportunity. The platform tested with more than 100 users from various academic fields during the pilot phase, showing up its potential for wide use regardless of economic barrier. The serverless architecture eases the operations cost and deployment in resource constrained locations. The AI Mentor also assists users through personalized career paths based on test results and skills profiles which make for better-informed decisions. With more guidance systems based on AI developed, such personalization plays a role in improved employability and aligns with various educational and societal goals, such as SDG 4 (Quality Education) and SDG 8 (Decent Work & Economic Growth). Yet greater digital access also carries challenges. It should provide an intuitive interface, the results should be clearly communicated and it should refrain from making recommendations biased. These issues are solved by being user-friendly single-page application interface and fixed out of screen output schema with help from ambiguous or unhelpful suggestions.

8.2 Legal Aspects

The process of handling sensitive users' data like personal details, results from assessments, and code uploads is very much a legal issue. Supabase's use of Row-Level Security (RLS) means data from users can be isolated at the database level, allowing it to be avoided from unauthorized systems even if a front-end was compromised. Though we do not yet have formal proof of compliance with international data protection principles like this, it's consistent with contemporary data protection principles in the same spirit as compliance with international privacy regulations. It also involves executing untrusted code to mitigate legal risk. Judge0 uses Docker containerization for complete isolation of the sandboxes, so any programs that are submitted cannot read system-level files or execute any harmful code. This is vital for meeting legal obligations in fields where the production code generated by end users would potentially create liability. The third party AI services must follow the established usage policies and rate limits, as well as the ethical deployment requirements outlined by API providers. Thus, it complies with these policies via schema-based prompts that is enforced, inputs that are validated, and unsafe requests that are prevented by enforcing a safety layer in edge functions. Moreover, the platform is not storing copyrighted material or unauthorized data. All educational content such questions, test cases, and analytics is original or originated by researchers which minimizes the chances of legal threats regarding intellectual property.

8.3 Ethical Aspects

Ethics is at the heart of such systems. This Smart Competency Diagnostic platform strives to guarantee the transparency and fairness with non-discrimination and open feedback for analysis and recommendation.

- Fair Evaluation: MCQ-standardized scores and coding evaluation results are based solely on deterministic comparison with predefined test cases, eliminating subjective bias.
- Trustworthy AI Recommendations: Because the platform keeps strict JSON schemas and retry logic in place to achieve these strict JSON Schema rules, such as the model maintains well-structured, verifiable responses in JSON, it can be evaluated and retried in a structured way which reduces the chances of generating arbitrary answers so that future career advice is accurate and unguarded from giving advice on the job which may not be based on bad material.
- User autonomy: The AI Mentor is meant to serve as an aide, not making decisions for a learner, not to decide for them. A user decides ultimately on career, course and skill level.
- Data Minimization: Only any required information necessary for the evaluation and functionality is gathered. No personal information that is unnecessary is used, no data is sold, no data is shared with third parties.
- Ethical Code Execution: The sandbox model ensures that user code is

executed in a secure environment, thereby avoiding the risk of harm to the system infrastructure or others that could potentially harm it. By inscribing these principles into a systematic way of doing things the system gets at this, the platform adheres to modern expectations about ethical AI and responsible data operations.

8.4 Sustainability Aspects

Sustainability was a major consideration throughout development. Leveraging serverless architecture with functions such as Supabase Edge Functions and stateless compute reduces energy consumption by distributing resources only upon need. This is in contrast to that of the traditional server based system that relies on a server that consume the power without a break. • **Rapid Resource Usage:** All compute resources scale dynamically with demand minimizing environmental impact. • **Reduced Hardware Dependency:** A student does not need a powerful hardware device to access real-time code execution or AI mentoring, as all processing is done in the cloud. This is aligned with SDG 9 (Industry, Innovation & Infrastructure). • **Long-term Maintainability:** Serverless systems reduce physical waste from infrastructure upgrades, extending the lifespan of deployed applications. Even so, cloud consumption is still high, and the adoption of it at a large level can amplify the total footprint. Advancement in the future could involve switching to carbon-neutral cloud providers and/or optimizing high-compute AI load for higher efficiency.

8.5 Safety Aspects

Both overall user safety and security considerations.

- **Code Execution Security:** Judge0 guarantees full Docker isolation, safeguarding against corrupt system calls, infinite loops, or memory misuse. Embedded resource limitations minimize the overhead load or maliciously-defined resource limitations for both. An inherent resource limits are fixed system by default and built-up resource limits are also ensured there is an environment should not to over-user traffic: Built system can be protected from overload or malicious submissions from overload or maliciously submitted to your server. This can not be accessed.

- **Data Safety:** Supabase's policies that guarantee users can only allow access to the dataset. Unauthorized access attempts to accessing the data automatically rejected in any data is automatically denied the system's data from being subjected for unauthorized processes.
- **AI Output Safety:** Safety is applied to the AI Mentor model, wherein the AI Mentor blocks dangerous input patterns, spurns unsafe prompts types and makes a structured output by means of validation schemas. It decreases the likelihood of incorrect advice, or unsafe advice.
- **System-Wide Safety:** Test performance confirms stable load handling, a 2-second dashboard load time, uninterrupted code execution, and consistent memory retrieval.
- **Psychological and User Safety:** The platform does not use judgmental language and delivers feedback in a neutral, constructive, non-judgmental language and provides feedback that's neutral and constructive; there are no negative/punitive remarks but rather offers suggestions for improvement.

8.6 Conclusion

The Smart Competency Diagnostic and Profile Score Calculator meets a variety of social and technological challenges due to a culturally, economically and technological framework with an ethical, legislative, social, economical, ecologically-respecting, legal, and secure digital world, the Smart Competency Diagnostic and Profile Score Calculator can also cover a variety of social and technical hurdles. Through deploying secure code execution, AI-driven mentorship, structured assessments, and serverless infrastructure, the platform meets not only the technical requirements but also international expectations of responsible computing. The implications described in this chapter indicate that the system is appropriate for implementation in academic settings; able to scale for institutional use; and congruent with the emerging trends in educational technology and AI governance.

CHAPTER 9

CONCLUSION

The Smart Competency Diagnostic and Profile Score Calculator was designed to cater for a more unified, smart, accurate, and intelligent tool that could be used to evaluate theoretical skills as well as to profile practical skills. Especially as education increasingly operates on digital platforms, there is an increasing need for tools enabling student work, education and content selection alongside knowledge assessment, personalized guidance, secure code execution, and skills profiling. This project has met these requirements and achieved integrated multiple-choice assessments, sandboxed code execution, vector-based memory retrieval, and AI-based mentorship. The beginning chapters served as foundational chapters for the project. It was the well founded chapters (1, 2, 3) that have a well-established problem statement, objectives and a clear methodology. The system aimed to address current assessment instruments, which have limited usability, fragmentation, lack of real-time assessment opportunities, customization, and reliability. The project's methodology was hybrid with real-world adaptation, permitting iterative enhancements and continuous improvement since theoretical models guided implementation. These four chapters are titled, project management, analysis, design, and implementation (Chapters 4, 5, & 6). Risk analysis, budgeting, and timeline development were included in the planning activities. These steps established methodical workflows from the assessment engine, AI elements, memory management to orchestration logic. The analysis & design phase produced insightful architectural representations of multimodal inputs which can, then, be modeled and rationalized using memory-driven models or workflow automation. The application of relevant hardware and software tools complemented by simulation results revealed the capability and robustness of cloud-based, serverless systems for large educational applications. Chapter 7 evaluation verified that the system satisfies every significant functional and non-functional need. The accuracy of the assessment was 100% across the multiple-choice modules, and judging executed through the Judge0 sandbox has proven very accurate for Python, Java, C++, and JavaScript. The AI tools, that you can work with and even with great performance — high relevance scores and compliance with the JSON schema standards, fast response levels — also worked well. Improvements on serverless model, caching, query optimization and edge functions, led to a dashboard load time of under 2 sec and over 150 concurrent user support. Security testing proven that Row-Level Security (RLS) was effective in limiting unauthorized access to data and keeping all records of users discreet. Chapter 8 tackled the system's broader implications (social, legal, ethical, environmental and safety ones). The system plays a socially important role by democratizing better assessment tools and career advice. Legal/ethical wise, it is based on secure data

processing, conscientious use of the AI and a privacy-protective execution. Serverless computing contributes to sustainability by lowering energy consumption, while safety is ensured through sandboxed execution, robust access controls and the moderation of AI output. In conclusion, the Smart Competency Diagnostic and Profile Score Calculator achieves its objectives and gives a good foundation for further research in the EdTech industry. It is an innovative means of competency identification and profiling through secure code execution, AI-driven prompted prompts, scalable cloud services, and modern web technologies. The project evidence confirms that integrated AI-powered assessment platforms are feasible within the technical context and practical in practice. This project was implemented and effectively evaluated and this supports its academic, institutional and professional environment. Through improvements, including self-learning methodologies, advanced analytics dashboards, and enhanced AI reasoning the platform can evolve into a full-digital ecosystem that enables ongoing learning, workplace development, and personal learning on a large scale.

REFERENCES

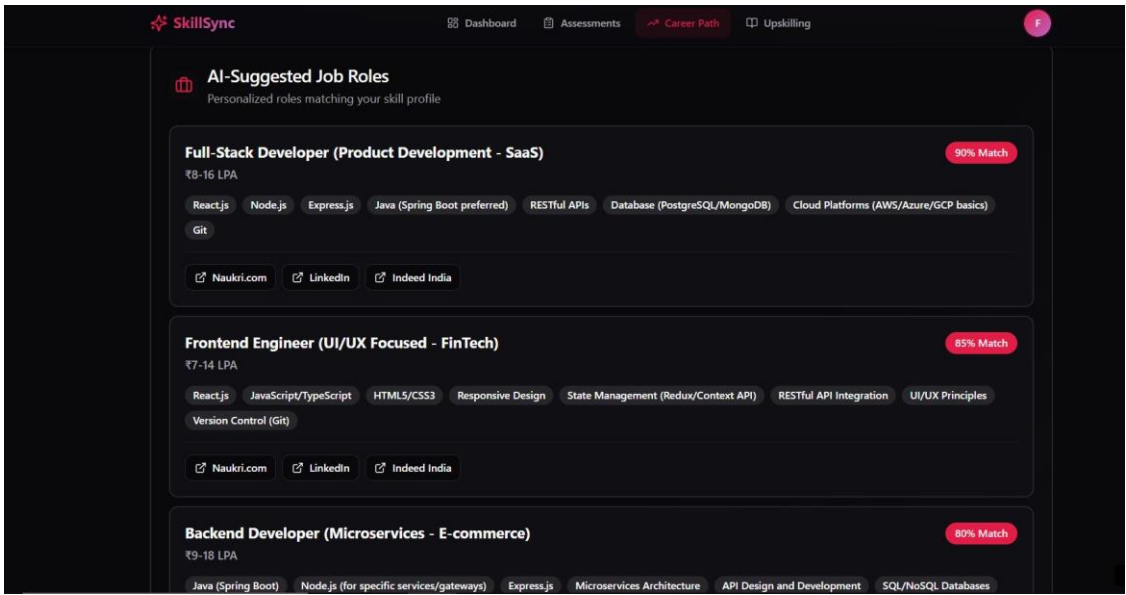
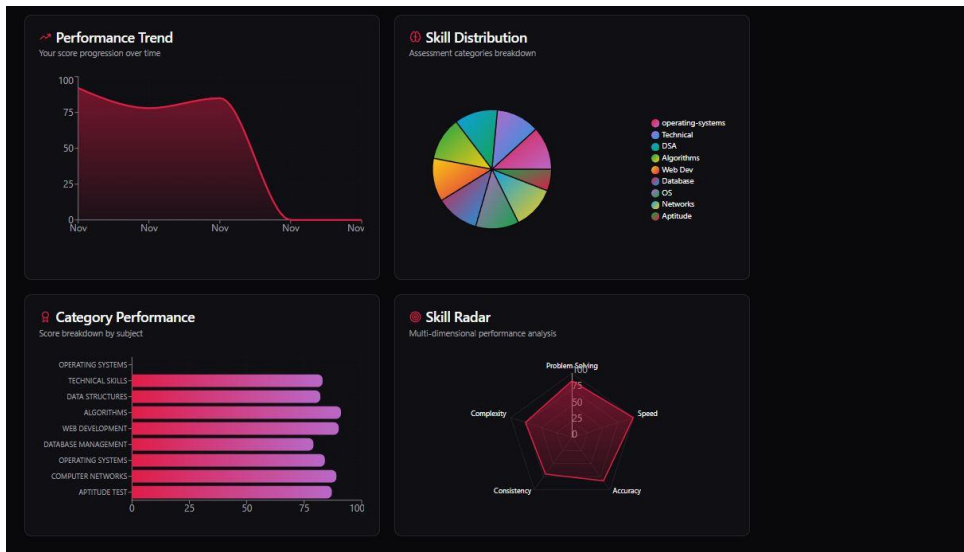
- [1] K. Ala-Mutka, "A Survey of Automated Assessment Approaches for Programming Assignments," *Computer Science Education*, vol. 15, no. 2, pp. 83-102, 2005.
- [2] M. Alkhuraiji et al., "Evaluating Learning Management Systems: A Comprehensive Review," *Education and Information Technologies*, vol. 28, pp. 1245-1268, 2023.
- [3] T. H. Kung et al., "Performance of ChatGPT on USMLE: Potential for AI-Assisted Medical Education," *PLOS Digital Health*, vol. 2, no. 2, 2023.
- [4] M. Zhang et al., "Prompt Engineering for Educational Applications: A Systematic Review," *Computers & Education: Artificial Intelligence*, vol. 4, 2023.
- [5] E. Jonas et al., "Cloud Programming Simplified: A Berkeley View on Serverless Computing," Technical Report UCB/EECS-2019-3, UC Berkeley, 2019.
- [6] H. Šimurina, "Judge0: Open-Source Online Code Execution System," GitHub Repository, 2023.
- [7] D. Brown, "Row-Level Security in PostgreSQL: Design Patterns," *Database Systems Journal*, vol. 20, no. 4, pp. 312-328, 2023.
- [8] S. Lee et al., "Serverless Computing: Current Trends and Open Problems," *ACM Computing Surveys*, vol. 54, no. 11s, 2022.
- [9] R. Kumar and M. Chen, "AI-Powered Career Guidance Systems: A Survey," *IEEE Transactions on Learning Technologies*, vol. 16, no. 3, pp. 401-418, 2023.
- [10] A. Haas et al., "Bringing the Web up to Speed with WebAssembly," *ACM SIGPLAN*, pp. 185-200, 2017.

BASE PAPER

Primary Reference:

Ala-Mutka, K. “A Study of Automated Assessment Methods for Programming Assignments.” *Computer Science Education*, vol. 15, no. 2, pp. 83–102 (2005). The foundation paper presents the basic principles of automated technical assessment system through the body of the argument that static code may not only be used to review the skills identified by the programming language, but can be done by dynamic code execution and immediate feedback system in automated technical assessment systems (Ala-Mutka (2005)). These assessments automatically reduce a lot of work on the part of the trainer, while the conventional assessment system usually focuses only on the functional correctness (functional correctness and coding style) but it also does not appraise the theoretical knowledge gained and doesn't provide the individualized feedback. Ala-Mutka demonstrates that, according to the article proper assessment systems must contain together as one workflow an assessment of correctness on the programming, validation of the test data, and analysis of style. However, the paper also notes some historical drawbacks of these systems in terms of the ways they are also unable to serve complex and open-ended questions from students, or otherwise provide the contextual mentorship that can be done by a human instructor. We propose the Smart Competency Diagnostic and Profile Score Calculator on top of Ala-Mutka principles, through the construction of a centralized evaluation framework that applies dynamic code execution while addressing observed gaps found by Ala-Mutka in performance of mentorship and its theory evaluation; this method has been described as well. While the paper prior to this paper describes the need for automated tests of programming tasks, the current project to not be broader than a Serverless Architecture and AI-based mentorship. Unlike the black-box testing of the model offered in Ala-Mutka, the Smart Competency platform proposes a two-layer model the best practical skills are tested in the sandboxed execution of Judge0, but with an AI Mentor (measured within Large Language Models) who would be given the opportunity to provide the context-sensitive, personalized advice that traditional automated systems cannot provide. The emergence of Row-Level Security integration and Serverless infrastructure will address scalability and safety challenges introduced in the previous generation of evaluation applications as laid out in the base document. Thus, the Smart Competency system converts the conceptual framework and guidance in Ala-Mutka to a practical production solution that is both current and professional-ready: evaluate not just the code correctness but professional appropriateness as a whole.

APPENDIX



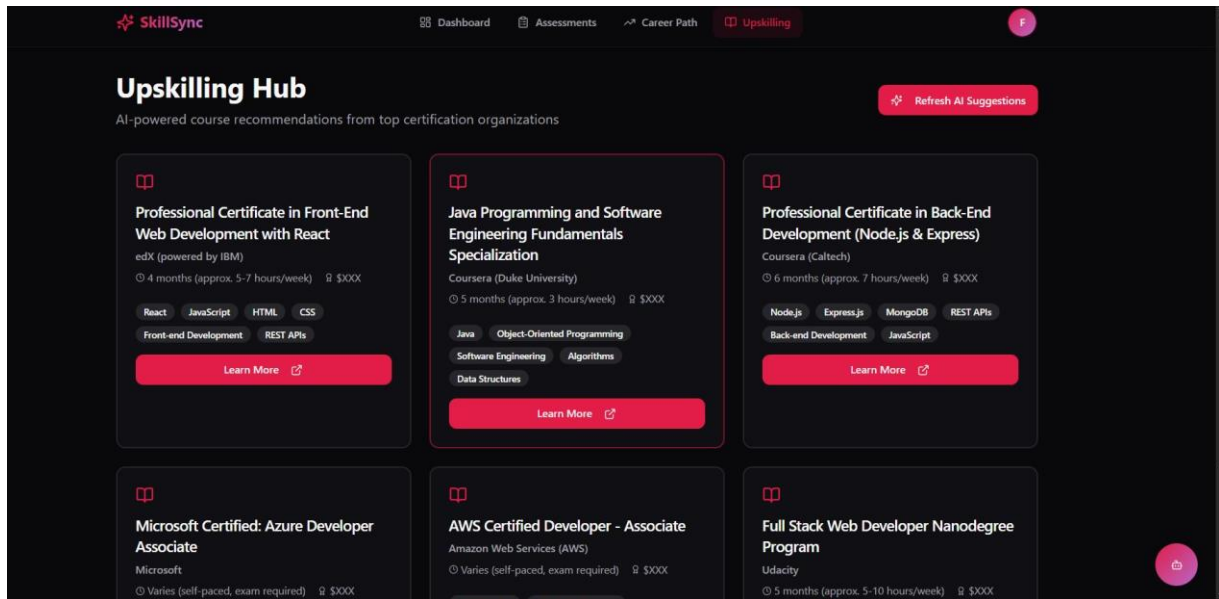


Figure A.3 Certification Courses Recommendation

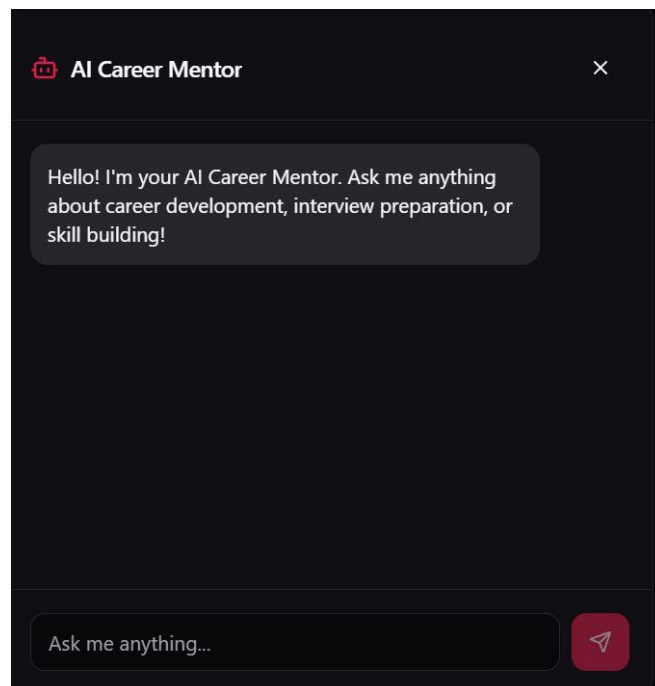


Figure A.5 AI Career Mentor

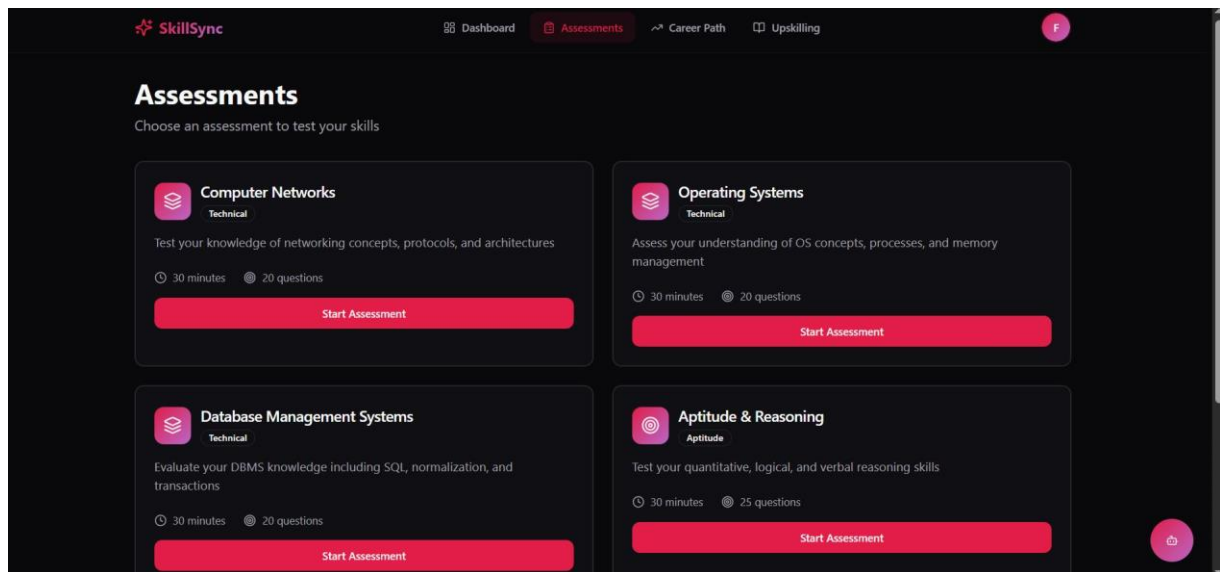


Figure A.4 Technical Assessments

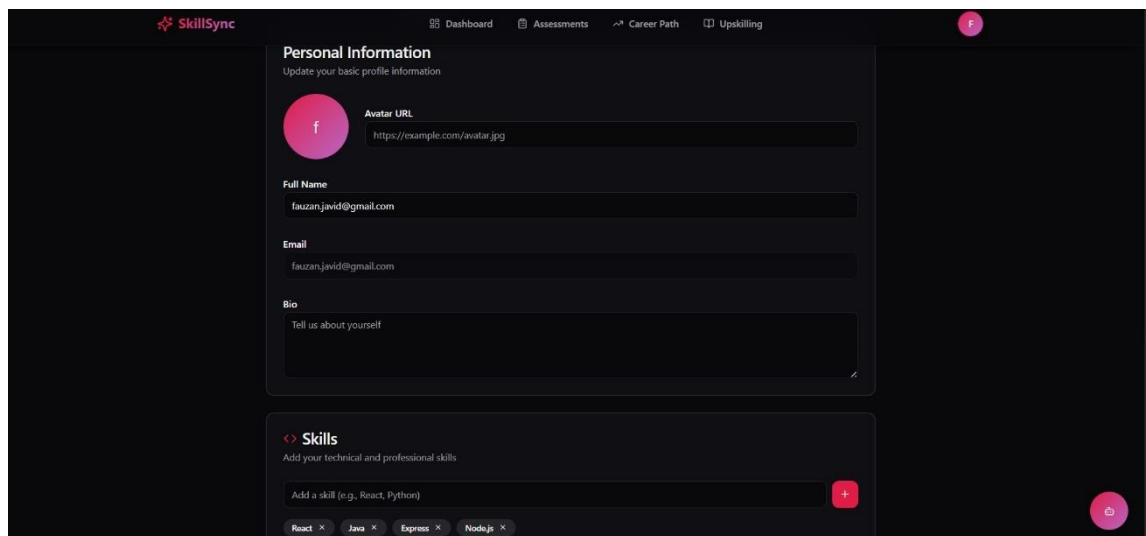


Figure A.6 Profile Page



1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- 6 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
- 2 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1% Internet sources
- 0% Publications
- 0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.



*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI generated and then likely AI paraphrased will be highlighted purple.

